



DEPARTEMENT INFORMATIQUE

RAPPORT DE PROJET DE FIN D'ÉTUDES

EN VUE DE L'OBTENTION DU :
DIPLOME NATIONAL D'INGÉNIEUR INFORMATIQUE
OPTION : GÉNIE LOGICIEL-ARCHITECTURE LOGICIELLE

Conception et développement d'une application de gestion d'un réseau professionnel de travaux de dépannage à domicile avec un système intelligent de vérification et de recommandation

Élaboré par : SALEM Wadia

Soutenu le 11/07/2023 devant le jury :

Président :	M. ACHOUR Sami	ISSAT de Sousse
Examinatrice :	Mme. KHEMAJA Maha	ISSAT de Sousse
Encadrante :	Mme.BELGACEM Selma	ISSAT de Sousse
Encadrant Industriel :	M. AZAIZ Yassine	Proxym IT

Année Universitaire : 2022-2023

Code Sujet : FI-GL23-004

Dédicace

“

Je suis reconnaissant envers tous ceux qui m'ont soutenu tout au long de ma vie.

*À mes chers parents, **Tijani** et **Latifa**, je vous remercie de tout cœur pour votre amour et votre soutien constants.*

*À mes frères, **Mokhles** et **Sameh**, je vous souhaite un avenir radieux.*

Je remercie également tous ceux qui me sont chers pour leur soutien inébranlable.

Enfin, un grand merci à toutes les personnes qui ont contribué à la réalisation de ce travail.

”

- Wadia

Remerciements

Je tiens à exprimer ma profonde gratitude envers les personnes exceptionnelles qui ont apporté leur précieuse contribution à la réalisation de ce travail. Je saisis cette occasion pour exprimer mes remerciements sincères et chaleureux à chacun d'entre eux.

Tout d'abord, je tiens à adresser mes remerciements les plus sincères à ma encadrante, Mme. Belgacem Selma. Sa compétence, sa patience et son encouragement ont été des moteurs essentiels dans la réussite de ce projet. Son regard critique avisé a joué un rôle déterminant dans la structuration du travail et l'amélioration de la qualité de chaque section. Je suis extrêmement reconnaissant de pouvoir bénéficier de sa précieuse expertise.

Un immense merci également à mon encadrant industriel, M. Azaiz Yassine, pour son soutien inestimable et son suivi attentif tout au long de cette expérience professionnelle. Ses conseils avisés et les informations qu'il m'a généreusement prodiguées ont été d'une valeur inestimable. Son professionnalisme remarquable a grandement contribué à enrichir mon apprentissage.

Je tiens à exprimer ma sincère gratitude envers mes enseignants de l'ISSATSo, dont l'expertise et le dévouement ont été une source d'inspiration constante tout au long de ma formation. Leur transmission de connaissances et leur volonté de nous pousser vers l'excellence ont été des éléments clés dans mon parcours académique.

Je souhaite également adresser mes remerciements les plus profonds aux membres du jury, qui ont accepté de consacrer leur temps et leur expertise pour évaluer mon travail. Leur engagement dans ce processus d'évaluation est un honneur que je leur suis extrêmement reconnaissant d'avoir accordé.

Enfin, je tiens à exprimer ma reconnaissance envers toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail. Votre soutien, vos conseils et votre encouragement ont été une source d'inspiration constante et m'ont permis d'atteindre ce résultat final avec détermination et professionnalisme.

Résumé

Le projet que nous décrivons dans ce rapport s'inscrit dans le cadre d'obtention du Diplôme National d'Ingénieur en Informatique, option Génie Logiciel, de l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse. Il est réalisé au sein de l'équipe DigiX "Front-End" de la société Proxy-mIT.

Face aux difficultés rencontrées lors de la demande de services de dépannage à domicile, nous avons mis en place, dans le cadre de ce projet, un réseau professionnel visant à fournir aux clients un moyen fiable et simple pour solliciter un service. Ce processus repose sur un mécanisme de recommandation, qui permet d'offrir aux clients le meilleur choix possible, tout en assurant une communication en temps réel entre le client et le professionnel.

Mots clés : Réseau professionnel, Recommandation, Machine Learning, Chat, Optical character recognition (OCR)

Abstract

The project described in this report was conducted in order to obtain the National Diploma of Engineer in Computer Science, Software Engineering option, from the Higher Institute of Applied Sciences and Technology of Sousse. It is being carried out within the DigiX "Front-End" team of ProxymIT company.

Faced with difficulties encountered when requesting home repair services, we established, as part of this project, a professional network aimed at providing customers with a reliable and simple means to request a service. This process is based on a recommendation mechanism, which allows offering customers the best possible choice while ensuring real-time communication between the customer and the professional.

Keywords : Professional network, Recommendation, Machine Learning, Chat, Optical character recognition (OCR)

Table des matières

Chapitre 1	Présentation du projet	3
Introduction		4
1.1	Présentation de l'organisme d'accueil	4
1.1.1	Des chiffres sur l'entreprise	4
1.1.2	Services	4
1.1.3	Les technologie adaptées	5
1.2	Présentation du projet	5
1.2.1	Cadre du projet	5
1.2.2	Problématique	5
1.2.3	Solution proposée	6
1.2.4	Objectifs	6
1.3	Étude de l'existant	6
1.3.1	Étude locale	6
1.3.2	Étude international	7
1.4	Processus de développement	7
1.4.1	Méthodologie adoptée	8
1.4.2	Incréments du logiciel	8
1.5	Chronologie	9
Conclusion		9
Chapitre 2	Spécification des besoins	11
Introduction		12
2.1	Identification des acteurs	12
2.2	Besoins Fonctionnels	12
2.3	Besoins non Fonctionnels	13
2.4	Diagrammes de cas d'utilisation	15
2.4.1	Diagramme de cas d'utilisation global	15
2.4.2	Raffinement des cas d'utilisation	16
2.4.2.1	Diagramme de demande d'un service	16
2.4.2.2	Diagramme de gestion des demandes de client	17
2.4.2.3	Diagramme de gestion de profil	18
2.4.2.4	Diagramme d'évaluation du travail réalisé	19
2.4.2.5	Diagramme d'administration de l'application	20
2.5	Diagramme d'activités	21
Conclusion		21
Chapitre 3	Conception	22
Introduction		23
3.1	Vue statique de l'application	23
3.1.1	Architecture globale du logiciel	23
3.1.2	Définition du patron Clean Architecture	24
3.1.3	Architecture du contrôleur traitements	25
3.1.4	Architecture du contrôleur GUI utilisateur	27

3.1.5	Architecture du contrôleur GUI d'administration	30
3.1.6	Module intelligent de recommandation	31
3.1.7	Module intelligent de vérification	31
3.2	Vue dynamique de l'application	32
3.2.1	Diagrammes de séquences	32
3.2.1.1	Créer un compte professionnel	33
3.2.1.2	Publier une demande de services	34
3.2.1.3	Gérer les demandes publiques des clients	35
3.2.1.4	Demander un service privé d'un professionnel	36
3.2.1.5	Gérer les demandes privées des clients	37
3.2.2	Diagramme d'états-transitions	37
Conclusion		37
Chapitre 4	Réalisation	39
Introduction		40
4.1	Technologies	40
4.1.1	Outils logiciels	40
4.1.2	Langage de développement	41
4.1.3	API et bibliothèques utilisée	41
4.2	Outils d'implémentation	41
4.3	Architecture physique	42
4.4	Interfaces de l'application	43
4.4.1	Lancement de l'application	43
4.4.2	Authentification	43
4.4.3	Gestion profile	44
4.4.4	Demander un service	46
4.4.5	Gestion des demandes des clients	46
4.4.6	Messagerie	47
4.4.7	Évaluation	48
4.4.8	Administration	48
Conclusion		50

Table des figures

1.1	Logo Proxym	4
1.2	Évolution de Proxym	4
1.3	Interface de recherche dans le site Jobber.tn	6
1.4	Résultat de recherche pour un service de bricolage dans le site petty-jobs.com	7
1.5	Cycle de la méthodologie évolutive [23, p. 19]	8
1.6	Diagramme de gantt du projet	9
2.1	Diagramme de cas d'utilisation globale	15
2.2	Diagramme détaillé de cas d'utilisation : demander un service	16
2.3	Diagramme détaillé de cas d'utilisation : gérer les demandes du client	17
2.4	Diagramme détaillé de cas d'utilisation : gérer profil	18
2.5	Diagramme détaillé de cas d'utilisation : évaluer le travail réalisé	19
2.6	Diagramme détaillé de cas d'utilisation : contrôler application	20
2.7	Diagramme d'activités partitionné des principales fonctionnalités	21
3.1	Diagramme de paquetages : Vue globale du logiciel	23
3.2	Diagramme d'architecture clean [26]	24
3.3	Diagramme de composants détaillé du sous-paquetage : Auth	25
3.4	Diagramme de classes de la couche Entités	26
3.5	Diagramme de paquetages détaillé : Contrôleur GUI utilisateur	28
3.6	Diagramme de classe du sous-paquetage : Core	29
3.7	Diagramme de composants détaillé d'une instance du Core	30
3.8	Diagramme de classes du paquetage : Contrôleur GUI d'administration	30
3.9	Diagramme de séquences détaillées modélisant la création d'un compte professionnel	33
3.10	Diagramme de séquences détaillées modélisant la demande publique d'un service	34
3.11	Diagramme de séquences détaillées modélisant la gestion des demandes publiques des clients	35
3.12	Diagramme de séquences modélisant la demande d'un service d'un professionnel spécifique	36
3.13	Diagramme de séquences modélisant la gestion de mes demandes reçues	37
3.14	Diagramme d'états-transitions pour une demande de services à domicile	37
4.1	Technologies utilisées	40
4.2	Diagramme de déploiement de notre application	42
4.3	Interfaces de lancement de l'application	43
4.4	Interfaces d'authentification	44
4.5	Interfaces de gestion profile	44
4.6	Interfaces des favoris	45
4.7	Interfaces de gestion profile professionnel	45
4.8	Interfaces de demander un service	46
4.9	Interfaces de gestion demandes des clients	47
4.10	Interfaces de messagerie	47
4.11	Interfaces d'évaluation	48
4.12	Interface d'authentification d'administration	48
4.13	Interface d'accueil d'administration	49

4.14 Interface des utilisateurs d'administration 49

4.15 Interface des types de service d'administration 50

Liste des tableaux

1.1	Comparaison des solutions présentées	7
2.1	Cas d'utilisation : demander un service	17
2.2	Cas d'utilisation : gérer les demandes des clients	18
2.3	Cas d'utilisation : évaluer le travail réalisé	19

Introduction générale

Les réseaux informatiques jouent un rôle crucial dans notre quotidien. Ces réseaux permettent la communication et l'échange d'informations à travers le monde, offrant une connectivité instantanée entre les individus. Les réseaux sociaux sont un exemple frappant de cette propagation des réseaux informatiques, ayant transformé notre façon d'interagir et de partager des informations.

Conscients de l'importance croissante de ces réseaux, nous avons entrepris le développement d'un réseau professionnel novateur, visant à fournir aux clients un accès accru à des services de dépannage à domicile (plomberie, électricité, jardinage...). Notre système repose sur un mécanisme de recommandation optimisé pour chaque évaluation de service effectuée. Les clients ont la possibilité de partager leur demande de manière privée avec un professionnel spécifique, ou de la rendre publique au sein de notre réseau.

Une fois la discussion initiée entre les parties et l'accord conclu via notre système de messagerie en temps réel, le professionnel peut procéder à l'exécution du travail demandé. À la suite de cette intervention, le client a la possibilité d'évaluer et de donner son avis sur la qualité du service reçu. Ces évaluations sont ensuite traitées par notre système de recommandation afin d'orienter les clients dans leurs choix de services. Ce système de recommandation utilise un mécanisme de prédiction basé sur une méthode de machine learning.

De plus, notre plateforme garantit un mécanisme intelligent de vérification des professionnels en utilisant des méthodes avancées telles que la détection des données à partir de la carte d'identité nationale en appliquant un mécanisme d'extraction de texte exploitant la technique d'OCR (Optical Character Recognition). Cela nous permet de garantir l'intégrité des professionnels présents sur notre réseau, offrant ainsi aux clients une expérience sécurisée et fiable.

Dans ce contexte, l'objectif de ce stage est la conception et le développement d'une application de gestion d'un réseau professionnel de travaux de dépannage à domicile avec un système intelligent de vérification et de recommandation.

Le projet que nous décrivons dans ce rapport s'inscrit dans le cadre d'obtention du Diplôme National d'Ingénieur en Informatique, option Génie Logiciel, de l'Institut Supérieur des Sciences Appliquées et de Technologie de Sousse pour l'année universitaire 2022/2023. Il est réalisé au sein de l'équipe DigiX "Front-End" de la société ProxymIT.

Le présent rapport rend compte du travail effectué dans le cadre de ce stage et est organisé en quatre chapitres structurés de la manière suivante :

Premièrement, le chapitre "**Présentation du projet**", ce chapitre introductif met en lumière l'entreprise d'accueil, en fournissant des informations contextuelles sur son domaine d'activité. Il expose également la problématique à laquelle le projet répond, en identifiant les défis et les lacunes existants. La solution proposée est présentée en détail, ainsi que les objectifs spécifiques qui ont été fixés pour ce projet. De plus, une étude de l'existant est réalisée, mettant en évidence les technologies ou les systèmes similaires déjà en place, et le processus de développement de l'application est décrit.

Le deuxième chapitre “**Spécification des besoins**” se concentre sur la définition des acteurs principaux de l’application développée et des besoins qu’elle doit satisfaire. Les besoins fonctionnels et non fonctionnels sont identifiés et spécifiés de manière détaillée. Les principales fonctionnalités de l’application sont également présentées, en mettant l’accent sur leur importance et leur contribution à la résolution de la problématique initiale.

Le troisième chapitre “**Conception**” présente la conception de l’application, en décrivant les schémas conceptuels et l’architecture adoptée pour la solution proposée. Les choix de conception et les différentes composantes de l’application sont expliqués en mettant en avant leur cohérence et leur pertinence par rapport aux besoins spécifiés. De plus, le comportement dynamique de l’application est décrit, permettant ainsi une compréhension approfondie de son fonctionnement.

Le quatrième chapitre “**Réalisation**” expose l’environnement et les outils de développement qui ont été utilisés pour concrétiser le projet. Les résultats obtenus sont présentés à travers les principales interfaces du logiciel, illustrant ainsi les fonctionnalités implémentées et leur apparence visuelle. Des détails techniques pertinents sont également fournis pour permettre une évaluation approfondie du travail réalisé.

En conclusion, ce rapport mettra en évidence les atouts de l’application développée, soulignant sa capacité à résoudre la problématique identifiée et à répondre aux besoins spécifiés. En outre, les perspectives d’amélioration de l’application seront également abordées, ouvrant ainsi la voie à de potentielles évolutions et itérations futures.

Chapitre 1

Présentation du projet

Introduction	4
1.1 Présentation de l'organisme d'accueil	4
1.1.1 Des chiffres sur l'entreprise	4
1.1.2 Services	4
1.1.3 Les technologie adaptées	5
1.2 Présentation du projet	5
1.2.1 Cadre du projet	5
1.2.2 Problématique	5
1.2.3 Solution proposée	6
1.2.4 Objectifs	6
1.3 Étude de l'existant	6
1.3.1 Étude locale	6
1.3.2 Étude international	7
1.4 Processus de développement	7
1.4.1 Méthodologie adoptée	8
1.4.2 Incréments du logiciel	8
1.5 Chronologie	9
Conclusion	9

Introduction

Nous commençons ce chapitre par la présentation de l'organisme d'accueil «ProxymIT». Ensuite, nous exposons le problème identifié ainsi que la solution proposée suivis par une étude de l'existant afin de déterminer la valeur ajoutée et de mettre en évidence les insuffisances. Enfin, nous concluons ce chapitre par la description de la méthodologie de gestion de notre projet.

1.1 Présentation de l'organisme d'accueil

ProxymIT est une entreprise tunisienne de développement basée à Sousse, active depuis 2006. Elle est aujourd'hui implantée sur 3 continents : Proxym IT Tunisie, Proxym France, Proxym Middle East et Proxym Egypte. Elle est considérée comme un leader de la région du Sahel dans le secteur des solutions convergentes liées au Web, au Mobile et au Système d'information, grâce à des atouts tels que l'indépendance, le goût du challenge et l'esprit d'équipe. ProxymIT propose des services numériques innovants dans des différents secteurs, tels que les secteurs bancaire, gouvernemental, les éditeurs et les startups.



FIGURE 1.1 – Logo Proxym

1.1.1 Des chiffres sur l'entreprise

Proxym regroupe plus que 200 ingénieurs qualifiés dans des différents domaines. Elle a complété avec succès plus de 400 projets digitaux et plus de 150 projets de système d'information.

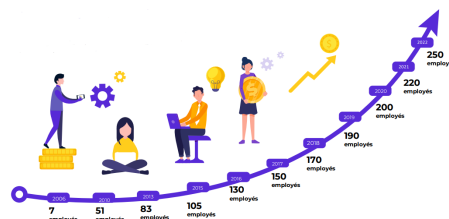


FIGURE 1.2 – Évolution de Proxym

1.1.2 Services

ProxymIT offre un catalogue de service évolutifs pour ses clients, comme suit :

- Digital Factory : C'est un service pour les startups qui vise à accélérer la livraison du produit et à contrôler l'avancement du projet.
- Gouvernements : Ce service est dédié aux gouvernements, villes ou institutions pour une évolution numérique. Proxym garantit l'optimisation des transformations digitales et construit une expérience digitale avancée par des applications.
- Banques : Proxym met à disposition une solution nommée Bankerise pour aider ses clients à rejoindre la digitale banque 3.0 afin de gagner une expérience personnalisée.
- Assurances : Insurise est une solution digitale d'assurance développée par Proxym pour digitaliser le domaine et faciliter les processus pour ses clients.

- Finance : Pour construire une relation forte avec les banques, Proxym offre le service Finance. Ce service garantit l'automatisation et l'accélération des transactions sans aucun coût supplémentaire dans un seul système.

1.1.3 Les technologie adaptées

ProxymIT dispose des compétences solides dans l'utilisation de différents technologies, telles que :

- Front-end : Cette pratique consiste à transformer les données en une interface graphique pour permettre à l'utilisateur de visualiser et d'interagir avec ces données par interaction numérique. Les outils utilisés comprennent AngularJS, ReactJS, VueJS ...
- Back-end : ce domaine de compétence concerne le développement et la gestion de la logique fonctionnelle de base et des opérations d'une application logicielle ou d'un système d'information. ProxymIT utilise notamment Symphony, SpringBoot, Nest ...
- Mobile : Le développement d'application mobiles consiste à créer des applications destinées aux appareils mobiles. Les technologies adoptés par ProxymIT sont Native Android avec Kotlin, Native IOS, Flutter, React-native ...
- Cloud DevOps Integrations : Cette compétence consiste à simplifier la connectivité entre les applications en permettant la communication entre les applications. Cette simplification est réalisée à l'aide de technologies telles que Docker, Amazon AWS ...

1.2 Présentation du projet

Nous détaillons dans cette section la problématique du projet, la solution proposée et les objectifs envisagés.

1.2.1 Cadre du projet

Les réseaux informatiques ont transformé la façon dont les individus interagissent et partagent des informations à travers le monde. Ils offrent un espace virtuel où les utilisateurs se connectent, partagent des idées et bénéficient de suggestions personnalisées grâce au système de recommandation intelligent utilisant un mécanisme de prédiction basé sur une méthode de machine learning. L'automatisation des tâches, telle que la vérification des données personnelles en appliquant un mécanisme d'extraction de texte utilisant la technique d'OCR (Reconnaissance Optique de Caractères), joue un rôle essentiel pour optimiser les processus et améliorer l'expérience utilisateur. Ce projet met l'accent sur les réseaux professionnel et explore les avantages de l'échange de données et de la recommandation intelligente.

1.2.2 Problématique

Monter un meuble, fixer une étagère, peindre une mure... parfois, le temps pour réaliser le travail est négligeable par rapport à la recherche d'un prestataire pour fournir ce service. Le travail de ce prestataire peut être malheureusement non satisfaisant, ce qui implique une perte de temps et d'argent. Dans le même contexte, le prestataire peut affronter des problèmes pour trouver des clients sérieux et des outils de travail performants et abordables.

Les questions qui se posent ainsi sont :

- Comment pouvons-nous optimiser la recherche d'un expert compétent pour un client ?
- Comment pouvons-nous fournir des renseignements correctes sur les compétences d'un prestataire pour un client ?
- Comment pouvons-nous fournir des opportunités d'affaires pour les professionnels ?
- Comment pouvons-nous fournir un moyen qui facilite le contact et la communication entre le client et le professionnel ?

1.2.3 Solution proposée

Pour résoudre ces problèmes détectés, nous proposons une application web/mobile de gestion d'un réseau de professionnels de travaux de dépannage à domicile (plomberie, électricité, jardinage...) et de leurs clients. Notre application doit faciliter le contact entre ces professionnels et leurs clients. En utilisant cette application, le client doit pouvoir rechercher d'une manière optimisée ces professionnels en accédant à des renseignements correctes et des avis de recommandations utiles sur les services fournis par ces professionnels. L'application doit également proposer aux professionnels des opportunités d'affaires et des occasions de travail.

1.2.4 Objectifs

Notre solution doit permettre de :

- Donner une description détaillée des services fournis par le professionnel.
- Recommander, avec un mécanisme intelligent, les experts adéquats selon les profils demandés par les clients.
- Identifier automatiquement les professionnels par un mécanisme de scan de leurs cartes d'identité utile en cas d'accidents de travail.
- Créer et gérer un agenda de travail pour les professionnels en ajoutant un mécanisme de notification.
- Planifier des rendez-vous d'intervention en coordination avec les clients.
- Fournir un moyen de communication en temps réel entre le client et le professionnel.
- Localiser l'expert le plus proche.
- Partager des astuces de bricolage.
- Évaluer les services fournis par le professionnel par un mécanisme de notation et de retour du feedback du client.

1.3 Étude de l'existant

Cette section représente une étude des solutions existantes sur le marché pour comprendre le domaine et dégager les insuffisances.

1.3.1 Étude locale

À l'échelle nationale en Tunisie, il n'y a pas aucune application mobile qui met en relation les professionnels et les clients pour des travaux à domicile. Cependant, il y a deux sites web ayant le même objectif :

1. jobber.tn : Il s'agit d'une plateforme web tunisienne qui permet l'échange de services. Malheureusement, elle est actuellement en panne, ce qui nous empêche d'effectuer des recherches.



FIGURE 1.3 – Interface de recherche dans le site Jobber.tn

2. petty-jobs.com : Il s'agit d'un site web tunisien qui se concentre principalement sur les services informatiques. Cependant, il ne fournit aucune fonctionnalité pour les interventions en bricolage.



FIGURE 1.4 – Résultat de recherche pour un service de bricolage dans le site petty-jobs.com

1.3.2 Étude internationale

A l'échelle internationale, il existe beaucoup d'applications pour réserver un prestataire pour tous les services du quotidien, mais ces applications ne fonctionnent pas en Tunisie. Nous introduisons les principales applications et nous comparons leurs caractéristiques à travers le tableau 1.1. Cette analyse permettra de dégager les lacunes de ce genre d'applications qui seront remédiées dans notre application.

- Yoojo : est une application mobile qui cible les professionnels européens à la recherche d'emplois. Elle fonctionne comme une plateforme qui connecte des personnes compétentes avec des entreprises et des organisations locales qui requièrent leurs talents.
- Thumbtrack : est une application qui s'adresse aux entrepreneurs et aux freelances. Elle offre une solution complète pour contrôler et suivre les revenus, les dépenses et la comptabilité de façon facile et performante.
- TaskRabbit : est une application utile qui permet aux utilisateurs de confier des tâches courantes ou spécifiques à des personnes qualifiées et disponibles. Que ce soit pour assembler des meubles, faire des courses, nettoyer la maison ou effectuer des travaux de réparation, TaskRabbit connecte des particuliers avec des "Taskers" habiles et disposés à aider.

Applications	Avantages	Inconvénients
Yoojo	Yoojo offre une messagerie pratique et un système d'évaluation du travail pour aider les utilisateurs à prendre des décisions éclairées.	Il n'est pas possible de localiser facilement le professionnel le plus proche et il manque une fonction d'enregistrement des professionnels précédemment sélectionnés.
Thumbtrack	Thumbtrack donne une recommandation pour faciliter le choix du professionnel.	Thumbtrack fonctionne seulement aux États-Unis.
TaskRabbit	Les utilisateurs de TaskRabbit peuvent laisser des avis et des commentaires sur les prestataires.	TaskRabbit facture des frais de service, ce qui peut augmenter le coût global des tâches demandées.

TABLE 1.1 – Comparaison des solutions présentées

1.4 Processus de développement

Le choix de la méthode de réalisation d'un projet est très important, car il influence le cycle de vie du projet qui peut avoir un impacte sur la qualité du système logiciel.

1.4.1 Méthodologie adoptée

Nous avons opté pour le modèle évolutif, qui se base sur un développement incrémental itératif axé sur la livraison quotidienne au client pour garantir une utilisation rapide des incréments du logiciel, et par la suite, permettre de découvrir les vrais besoins du client plus tôt dans le cycle de vie du logiciel et ainsi améliorer au fur et à mesure sa qualité.

En effet, le modèle évolutif garantit une :

- Livraison rapide de produits fonctionnels : le modèle décompose le projets en incréments fonctionnels développés en ordre de priorité pour permettre une livraison rapide utilisable.
- Meilleure collaboration avec le client : Cette approche permet une communication régulière et des retours de feedback pour l'ajout de nouvelles fonctionnalités au cours du temps.
- Flexibilité : La méthode évolutive nous donne une flexibilité en fonction des besoins du client.

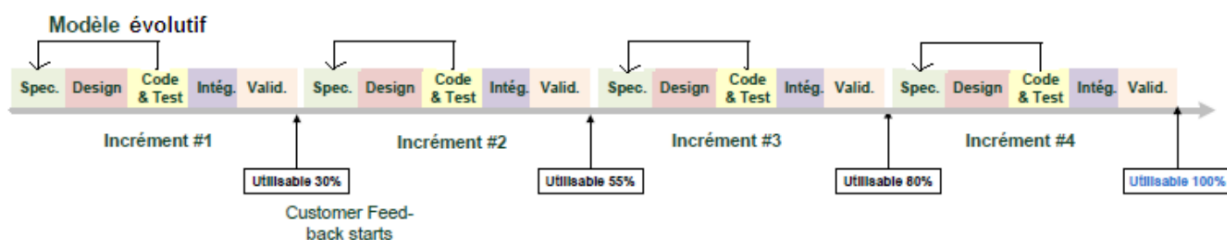


FIGURE 1.5 – Cycle de la méthodologie évolutive [23, p. 19]

Il est important que chaque incrément livré apporte une valeur ajoutée à la version globale du logiciel en ajoutant une fonctionnalité réelle et souhaitée par le client.

1.4.2 Incréments du logiciel

Les incréments de notre application sont :

- **Authentification** : en choisissant un rôle de client ou de professionnel, le visiteur peut remplir un formulaire et créer son compte. Ensuite, il pourra utiliser ses données de connexion pour se connecter.
- **Vérification du compte professionnel** : notre application doit vérifier les données personnelles du professionnel suite au scan de sa carte d'identité nationale, et ce traitement sera effectué par notre système intelligent de vérification.
- **Gérer profile** : Nous commencerons par la modification des données personnelles à partir de l'interface de profil pour tous les utilisateurs. Ensuite, nous ajouterons la possibilité pour le professionnel de gérer ses compétences et ses disponibilités, et nous terminerons en permettant au client de gérer sa liste de favoris.
- **Demander service** : cette partie est principalement pour le client où il peut consulter les professionnels en fonction de son besoin. Le client demande un service à un professionnel spécifique en privé ou de publier la demande à tous les prestataires proches et d'étudier les offres proposées.
- **Gérer les demandes de clients** : chaque professionnel reçoit des demandes privées et peut rechercher des demandes similaires publiées qu'il peut accomplir.

- **Messagerie** : cette partie est un module de chat en temps réel entre le client et le professionnel.
- **Évaluation** : après la réalisation du travail, le client a la possibilité de donner son avis par un commentaire et un nombre d'étoiles entre 1 et 5.
- **Recommandation des professionnels** : en se basant sur l'évaluation donnée, notre mécanisme intelligent de recommandation traite l'avis du client et attribue un score aux professionnels pour les trier.
- **Partage des astuces de bricolage** : notre application encourage la publication et l'échange de conseils de bricolage grâce un système d'articles.
- **Administration** : contrôle de l'application (contrôle des comptes utilisateur et gestion des types de services disponibles...).

1.5 Chronologie

Le diagramme de gantt est fréquemment utilisé pour la gestion de projet et pour représenter l'état d'avancement. La figure 1.6 représente la planification temporelle de notre projet en utilisant le diagramme de Gantt (l'unité de mesure est le jour).

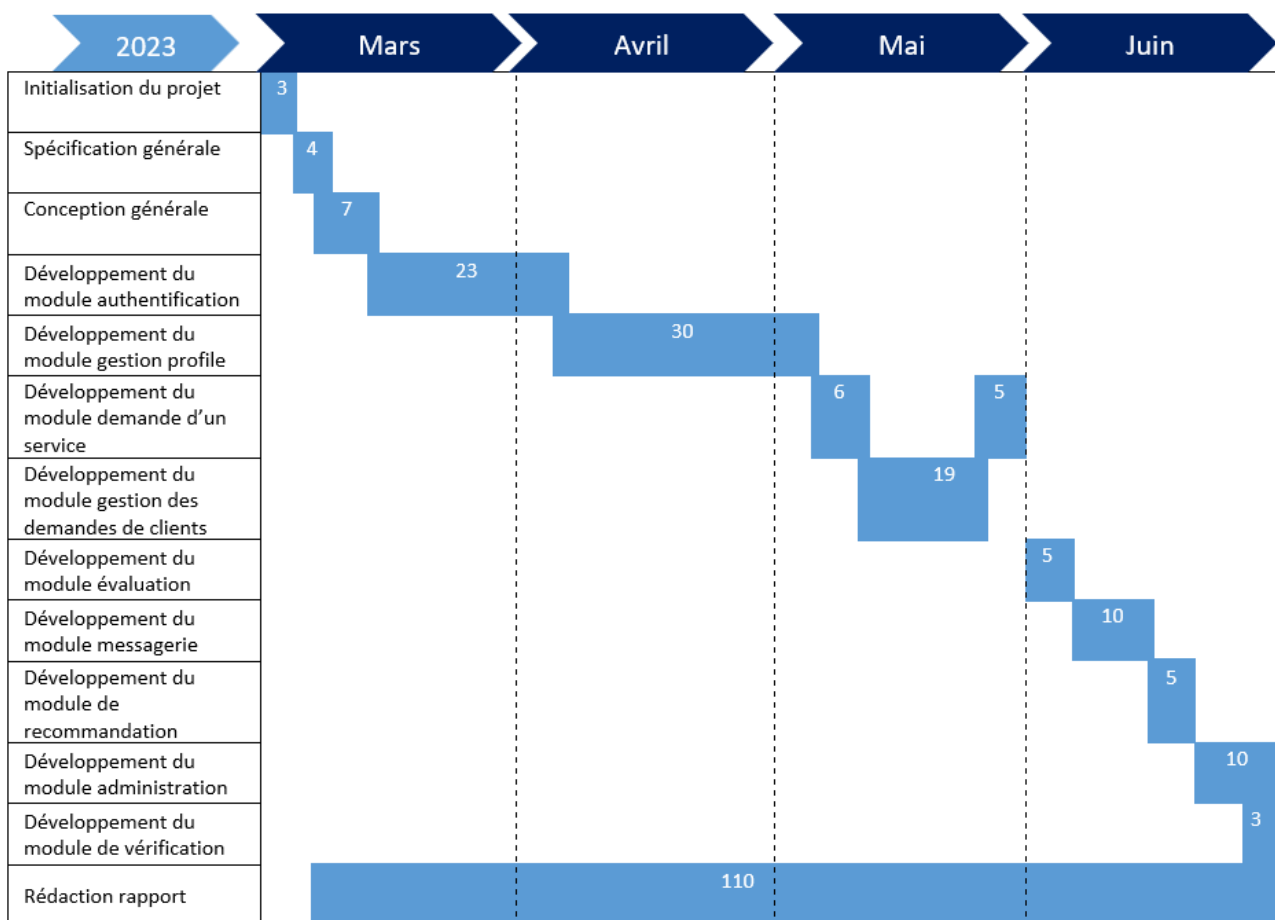


FIGURE 1.6 – Diagramme de gantt du projet

Conclusion

A travers ce chapitre, nous avons présenté l'organisme d'accueil ProxymIT. Nous avons également exposé la problématique et la solution proposée. Une étude de l'existant est aussi incluse dans ce chapitre. Nous avons enfin défini le processus de développement du logiciel adopté et son application. Dans le prochain chapitre nous décrivons les fonctionnalités que notre logiciel doit fournir pour répondre aux besoins du client.

Chapitre 2

Spécification des besoins

Introduction	12
2.1 Identification des acteurs	12
2.2 Besoins Fonctionnels	12
2.3 Besoins non Fonctionnels	13
2.4 Diagrammes de cas d'utilisation	15
2.4.1 Diagramme de cas d'utilisation global	15
2.4.2 Raffinement des cas d'utilisation	16
2.4.2.1 Diagramme de demande d'un service	16
2.4.2.2 Diagramme de gestion des demandes de client	17
2.4.2.3 Diagramme de gestion de profil	18
2.4.2.4 Diagramme d'évaluation du travail réalisé	19
2.4.2.5 Diagramme d'administration de l'application	20
2.5 Diagramme d'activités	21
Conclusion	21

Introduction

Dans ce deuxième chapitre, nous nous intéresserons à la spécification des besoins de notre projet. Nous commencerons par identifier les besoins fonctionnels et non fonctionnels nécessaires à la réalisation de notre application exprimés par le client. Nous préciserons également les différents acteurs qui interviennent dans notre projet et les exigences spécifiques liées à chacun d'eux. Enfin, nous utiliserons les diagrammes de cas d'utilisation et les diagrammes de séquences système pour expliquer davantage les fonctionnalités de notre application et les interactions entre les différents acteurs et notre application.

2.1 Identification des acteurs

Les acteurs de notre projet sont à la fois des utilisateurs humains et des systèmes informatiques qui interagissent directement avec l'application. Nous pouvons classer ces acteurs en deux catégories : les acteurs principaux, qui déclenchent des fonctionnalités au niveau de l'application, et les acteurs secondaires, qui fournissent des informations complémentaires pour permettre le bon fonctionnement de l'application. Cette classification des acteurs nous permet de mieux comprendre leurs rôles et d'adapter l'interface et les fonctionnalités de l'application à leurs besoins spécifiques.

Acteurs principaux :

- **Visiteur** : Il peut parcourir les profils des professionnels et les astuces de bricolage publiées sur la plateforme, mais il ne peut pas interagir avec ces professionnels ni demander de services, car ces fonctionnalités nécessitent une connexion.
- **Client** : il peut demander un service à domicile en invitant un professionnel spécifique en privé ou en publiant sa demande en public pour que des professionnels localement proches puissent proposer leurs services.
- **Professionnel** : il s'agit du prestataire qui fournit des services de dépannage à domicile et peut consulter les demandes des clients publiées sur l'application.
- **Administrateur** : il a pour rôle de gérer les services de l'application, de consulter les utilisateurs et les réclamations, ainsi que de garantir le bon fonctionnement de l'application.

Acteurs secondaires :

- **Système de vérification** : pour vérifier l'identité des professionnels, actuellement ce système a pour rôle de comparer les données saisies par le professionnel et les données extraites à partir de sa carte d'identité.

2.2 Besoins Fonctionnels

Le but du projet est de créer une application de gestion d'un réseau professionnel de travaux de dépannage à domicile qui doit répondre aux besoins fonctionnels des utilisateurs.

Dans ce projet les besoins fonctionnels sont :

- **Authentification** : il s'agit d'un module qui regroupe la création d'un compte et la connexion dans le but de sécuriser et tracer les actions de chaque utilisateur.

- **Gestion profil** : ce module est un élément essentiel pour assurer une expérience utilisateur personnalisée et de qualité. Il permet aux utilisateurs de modifier leurs paramètres personnels, de gérer les demandes et de contrôler les notifications qu'ils reçoivent.
 - **Pour les professionnels** ce module permet également à l'aide du système de vérification précédemment introduit de vérifier le compte, afficher l'état et les statistiques des travaux de dépannage réalisés et mettre à jour le profil professionnel.
 - **Pour les clients** : ce module permet aussi de gérer la liste des favoris.
- **Demande d'un service** : ce module est dédié aux clients qui souhaitent demander un service à domicile en fonction de leurs besoins.
 - **Demande privée** : pour consulter un professionnel spécifique.
 - **Demande publique** : il s'agit d'une demande publiée sur le réseau des professionnels sous forme d'un article. Les professionnels disponibles et localement proches peuvent répondre à cette demande et offrir leurs services. Ensuite, le client coordonne avec le professionnel choisi pour réaliser le service souhaité.
- **Gestion des demandes de clients** : ce module est dédié aux professionnels.
 - **Demande privée** : A la réception d'une demande privée d'un client, le professionnel peut accepter ou refuser la demande.
 - **Les demandes publiques** : Les professionnels peuvent consulter toutes les demandes publiques sur le réseau et choisir de répondre aux demandes d'intervention faisant partie de leur domaine de compétences et localement accessibles.
- **Communication entre client et professionnel** : Il s'agit d'un module de messagerie en temps réel qui permet une communication entre le demandeur de service et le prestataire. Cette communication n'est possible qu'après avoir répondu à une demande publique ou après l'envoi d'une demande privée.
- **Évaluation du travail** : Pour permettre à notre application de proposer des recommandations fiables des professionnels, il est préférable, qu'après chaque intervention réalisée, le client donne une évaluation du travail effectué.
- **Administration de l'application** : Pour contrôler l'application, l'administrateur peut gérer les services disponibles. Il peut gérer aussi les comptes des utilisateurs avec la possibilité de les bloquer/débloquer en se basant sur les réclamations reçues.

2.3 Besoins non Fonctionnels

Les besoins non fonctionnels sont les caractéristiques d'une application qui se basent sur les contraintes à prendre en considération afin d'assurer une performance et une qualité optimales. Ils comprennent des critères tels que l'ergonomie, la compatibilité, la sécurité et la rapidité. Ces critères sont essentiels pour fournir une application qui répond aux attentes des utilisateurs finaux et qui répond aux objectifs.

Dans ce projet les besoins non fonctionnels sont :

- **Ergonomie** : L'application doit être simple à manipuler pour tout âge d'utilisateur et doit être téléchargeable sur les smartphones et les tablettes.

- **Compatibilité** : Notre application est compatible avec tous les smartphones équipés d'un système d'exploitation Android de la version 8 (également connue sous le nom d'Oreo) ou supérieure, ce qui représente environ 90,7% des appareils sur le marché.
- **Sécurité** : Pour garantir un niveau de sécurité élevé, notre application doit vérifier les opérations suivantes :
 - Demander un nombre minimal d'autorisations de la part de l'utilisateur lors de l'installation ou de l'utilisation de la fonctionnalité.
 - Si l'utilisateur demande l'accès à des informations privées, exiger un code PIN/mot de passe/modèle ou un identifiant biométrique
 - Les données de l'application sont sécurisées dans des web service. Pour communiquer entre l'application et le web service les données transmises sont cryptées en utilisant TLS trafic avec une autorité de certification personnalisée pour confirmer la communication.
 - Pour identifier chaque utilisateur et contrôler ses actions un token d'accès doit être présent dans les requêtes envoyées.
 - Dans les deux parties Front-end et Back-end, vérifier et valider les données saisi avant l'exécution.
- **Rapidité** : Le temps de réponse de notre application doit être minimisé en appliquant la technique d'enregistrement quotidien d'une certaine quantité d'information en mémoire locale de l'appareil mobile et la technique de rafraîchissement de l'interface seulement en cas de nécessité d'affichage de nouvelles information.

2.4 Diagrammes de cas d'utilisation

Dans cette partie, nous présentons les fonctionnalités dégagées à partir des besoins fonctionnels sous forme des diagrammes de cas d'utilisation (ou "Use Case Diagrams" en anglais).

2.4.1 Diagramme de cas d'utilisation global

La figure 2.1 illustre le diagramme de cas d'utilisation global de notre application.

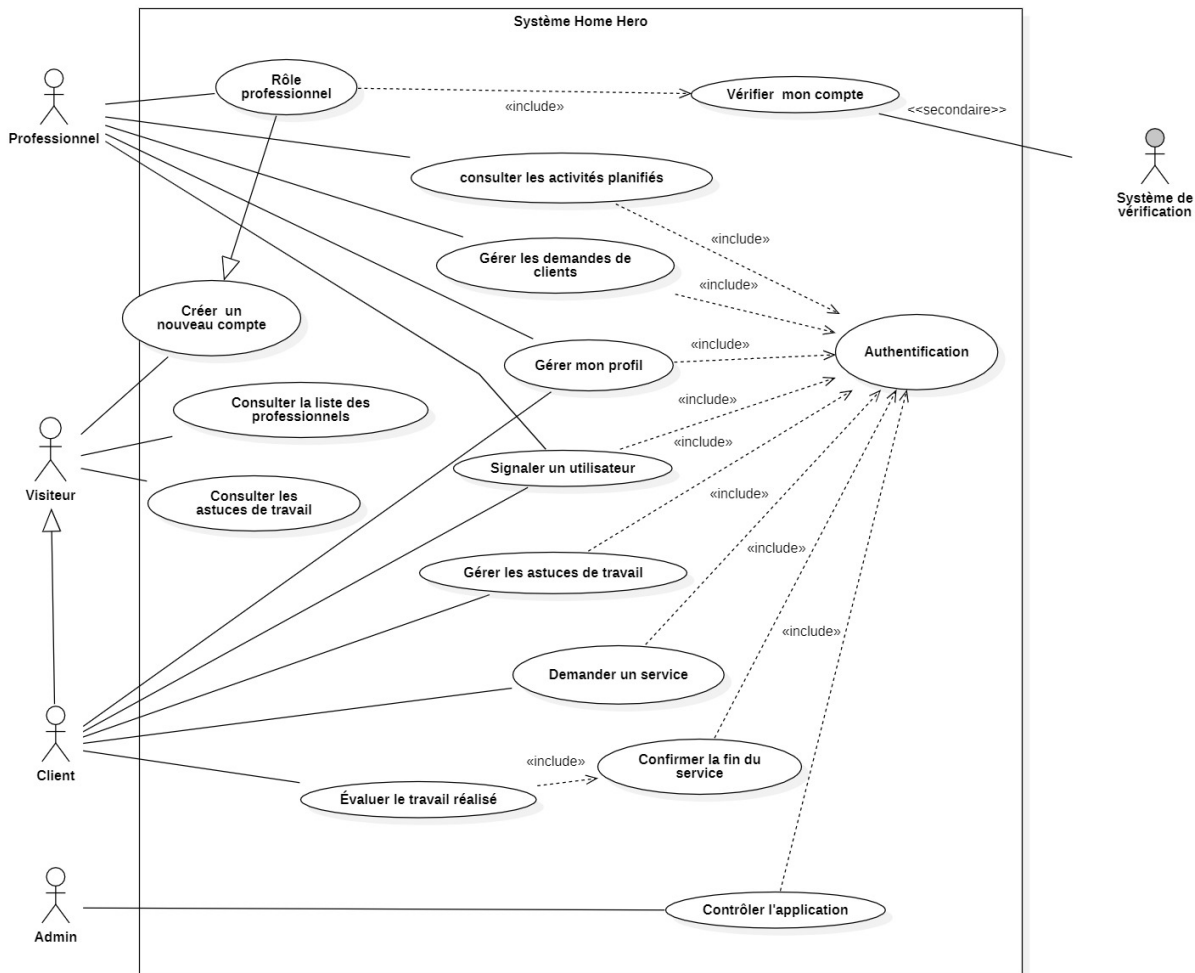


FIGURE 2.1 – Diagramme de cas d'utilisation globale

La figure 2.1 illustre les différentes fonctionnalités et actions principales accessibles aux utilisateurs. En nous basant sur ce schéma, nous pourrions décrire de manière concise et précise les cas d'utilisation les plus pertinents :

- **Créer un nouveau compte** : Chaque utilisateur de notre application peut créer son propre compte.
- **Vérifier compte** : L'application doit vérifier le compte du professionnel nouvellement créé à l'aide de notre mécanisme de vérification.
- **Demander un service** : Pour demander un service, il est nécessaire d'avoir un compte client authentifié, ce cas d'utilisation sera détaillé dans la section 2.4.2.1.
- **Gérer les demandes de clients** : Le professionnel doit être authentifié pour la gestion des demandes de clients, ce cas d'utilisation sera détaillé dans la section 2.4.2.2.

- **Consulter les activités planifiées** : Le professionnels peut déclencher une fonctionnalité qui l'aide à consulter ses activités planifiées pour ne pas rater aucun service.
- **Gérer mon profil** : Pour une expérience personnalisée, les professionnels et les clients peuvent gérer leurs comptes, ce cas d'utilisation sera détaillé dans la section 2.4.2.3.
- **Évaluer le travail réalisé** : Après la confirmation de la fin du service, le client peut évaluer le travail réalisé utile pour la recommandation automatique des professionnels. Ce cas d'utilisation sera détaillé dans la section 2.4.2.4.
- **Consulter la liste des professionnels** : Les visiteurs et les clients peuvent accéder à la liste des professionnels offrant le type de service demandé. Les professionnels sont classés selon le score attribué par notre mécanisme de recommandation.
- **Gérer les astuces de travail** : Notre application permet le partage des astuces de bricolage entre les clients.
- **Signaler un utilisateur** : Les utilisateurs authentifiés peuvent exprimer les problèmes rencontrés avec d'autres utilisateurs.
- **Contrôler l'application** : cette fonctionnalité est accessible uniquement par l'administrateur pour gérer les types de services disponibles dans notre plateforme et gérer les comptes utilisateurs, ce cas d'utilisation sera détaillé dans la section 2.4.2.5.

2.4.2 Raffinement des cas d'utilisation

Dans cette section, nous allons examiner en détail les cas d'utilisation les plus importants.

2.4.2.1 Diagramme de demande d'un service

La première fonctionnalité qui doit être valorisée est la demande d'un service qui est destinée uniquement aux clients.

La figure 2.2 illustre le diagramme de cas d'utilisation "demander un service".

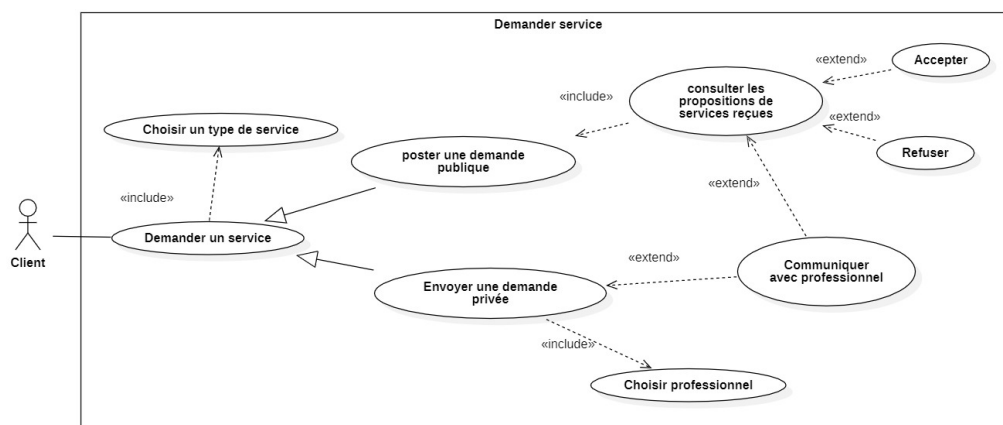


FIGURE 2.2 – Diagramme détaillé de cas d'utilisation : demander un service

Description textuelle :

Pour demander un service, le client doit d'abord sélectionner le service souhaité dans notre application. Il y a deux façons pour le client de trouver le professionnel idéal pour répondre à ses besoins. Dans le premier cas, il peut sélectionner un professionnel spécifique à partir d'une liste de prestataires recommandés par notre plate-forme, qui inclut une description de chacun pour faciliter le choix. Dans le deuxième cas, il peut décrire son besoin et laisser les professionnels proposer leurs services. Finalement, le client peut communiquer avec les experts sélectionnés pour planifier et finaliser les détails.

Description textuelle	
Titre	Demander un service
Acteur primaire	Client
Pré-condition	Client authentifié
Post-condition	Demande satisfaite
Scénario nominal	#Demander à un professionnel spécifique 1. Sélectionner le service à demander. 2. Choisir d'afficher les professionnels spécialisés. 3. Sélectionner un prestataire de la liste consulté. 4. Remplir les informations nécessaires concernant le besoin. 5. Le professionnel accepte l'invitation reçue.
Scénario alternatif	#Publier la demande 1. Sélectionner le service à demander. 2. Choisir de publier la demande. 3. Remplir les informations nécessaires concernant le besoin. 4. Le professionnel propose ses services pour cette demande. 5. Le client accepte la proposition.
Scénario d'exception	Le professionnel refuse l'invitation privé / Le client refuse les propositions reçues

TABLE 2.1 – Cas d'utilisation : demander un service

2.4.2.2 Diagramme de gestion des demandes de client

Le deuxième cas d'utilisation principal est la gestion des demandes de la part des professionnels, détaillé dans la figure 2.3.

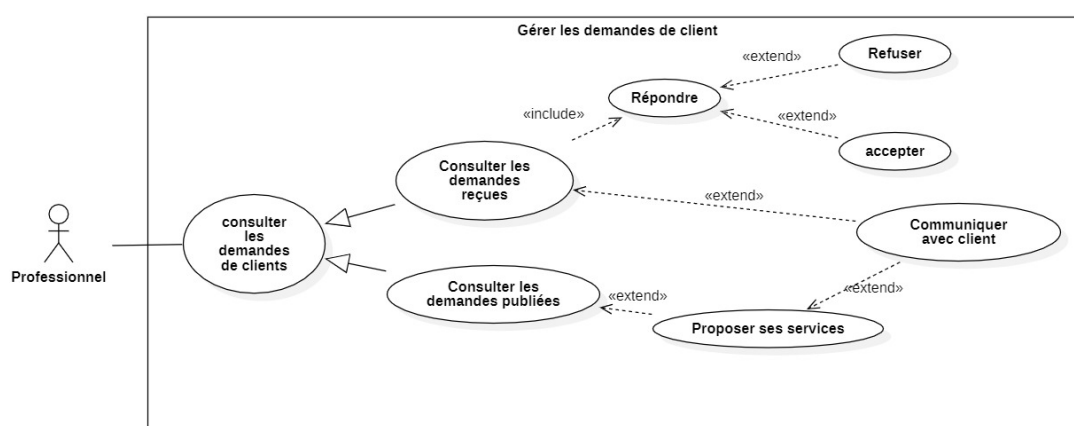


FIGURE 2.3 – Diagramme détaillé de cas d'utilisation : gérer les demandes du client

Description textuelle :

Pour gérer les demandes, les professionnels choisissent de consulter les demandes reçues en privé ou publiées selon leurs compétences. Pour la première option, ils ont la possibilité d'accepter ou de refuser la demande. Dans la deuxième option, les prestataires choisissent les opportunités de travail auxquelles

ils postulent en visant l'acceptation finale du client.

Description textuelle	
Titre	Gérer les demandes des clients
Acteur primaire	Professionnel
Pré-condition	Professionnel authentifié et vérifié
Post-condition	Demande acceptée
Scénario nominal	#Consulter les demandes privées 1. Afficher la liste des demandes privées reçues. 2. Accepter une demande privée.
Scénario alternatif	#Consulter les demandes publiées 1. Afficher la liste des demandes publiées. 2. Choisir une demande. 3. Proposer un service. 4. Le client accepte le service.
Scénario d'exception	Le client refuse service proposé / le professionnel refuse la demande privée reçue

TABLE 2.2 – Cas d'utilisation : gérer les demandes des clients

2.4.2.3 Diagramme de gestion de profil

Chaque compte client ou professionnel nécessite un contrôle et des opérations d'édition. Ces opérations diffèrent pour un compte client et pour un compte professionnel. La figure 2.4 illustre cette différence.

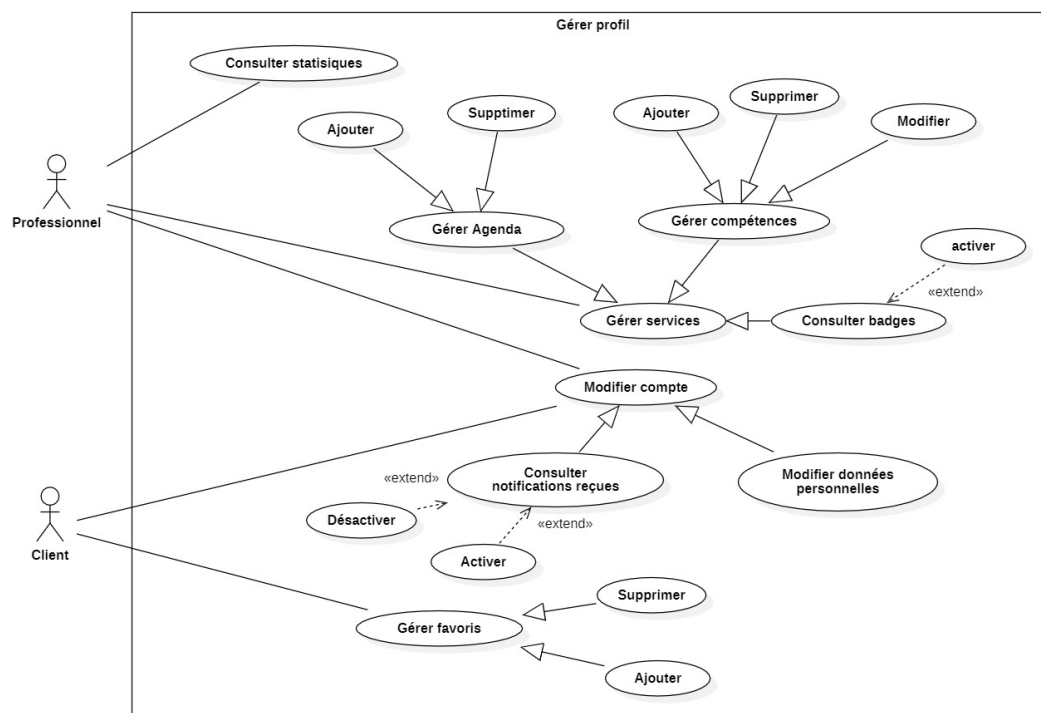


FIGURE 2.4 – Diagramme détaillé de cas d'utilisation : gérer profil

Description textuelle :

Pour la gestion du profil, nos utilisateurs ont la possibilité de modifier leur compte, de gérer leurs données personnelles et de consulter les notifications reçues. Ils ont également la possibilité d'activer ou de désactiver ces notifications selon leurs préférences. D'une part, le client peut ajouter ou supprimer des professionnels de sa liste de favoris. D'autre part, le professionnel peut consulter ses statistiques sur ses services (nombre de services, types de services, taux d'acceptation...), gérer son agenda en fixant ses horaires de travail, gérer ses compétences et enfin consulter les badges d'excellence qu'il a gagnés.

2.4.2.4 Diagramme d'évaluation du travail réalisé

La figure 2.6 représente les actions déclenchées par le client et accomplies par le système pour évaluer le travail réalisé..

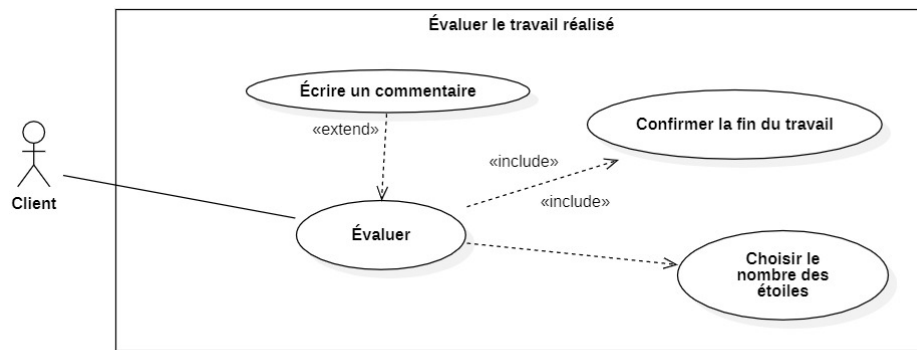


FIGURE 2.5 – Diagramme détaillé de cas d'utilisation : évaluer le travail réalisé

Description textuelle :

Après la fin du service demandé, le client a la possibilité d'évaluer le travail réalisé en attribuant un nombre d'étoiles compris entre 1 et 5, et en ajoutant un commentaire. Ce commentaire sera traité par notre mécanisme de recommandation une fois l'évaluation confirmée.

Description textuelle	
Titre	Évaluer le travail réalisé
Acteur primaire	Client
Pré-condition	Travail terminé
Post-condition	Travail évalué
Scénario nominal	#Évaluation avec commentaire 1. Sélectionner le service à évaluer. 2. Sélectionner le nombre d'étoiles. 3. Écrire un commentaire. 4. Confirmer l'évaluation.
Scénario alternatif	#Évaluation sans commentaire 1. Sélectionner le service à évaluer. 2. Sélectionner le nombre d'étoiles. 3. Confirmer l'évaluation.
Scénario d'exception	Un nombre d'étoiles non compris entre 1 et 5

TABLE 2.3 – Cas d'utilisation : évaluer le travail réalisé

2.4.2.5 Diagramme d'administration de l'application

L'administrateur de l'application a pour rôle de contrôler le bon fonctionnement de l'application. La figure 2.6 illustre les actions qu'il peut entreprendre.

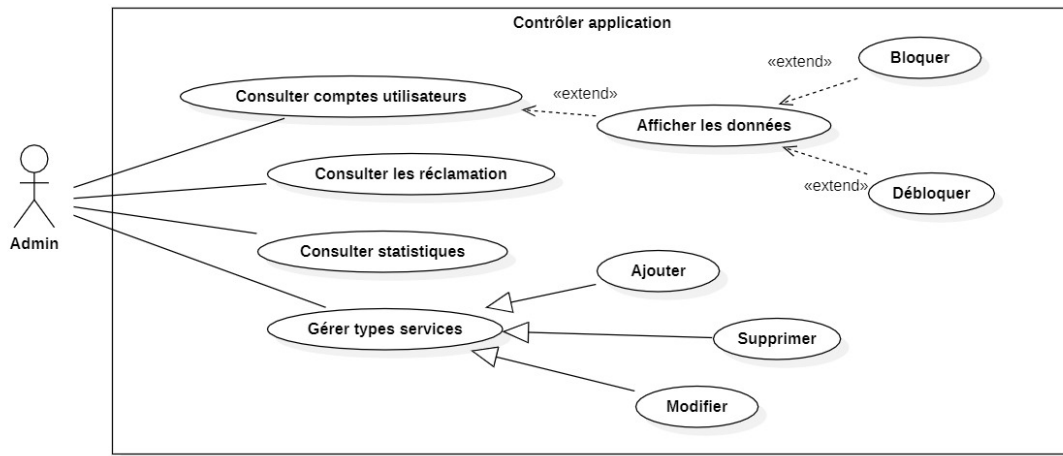


FIGURE 2.6 – Diagramme détaillé de cas d'utilisation : contrôler application

Description textuelle :

Notre application est dédiée à la gestion des services à domicile que chaque client peut demander. Dans cette application, l'administrateur gère l'ajout, la modification et la suppression de ces services. Il a également la possibilité de consulter les compte des utilisateurs. Il peut les bloquer/débloquer en cas d'incidents en fonction des remarques reçues (signalées), tels qu'un problème lors du déroulement du travail ou un manque de ponctualité. Enfin, cet acteur consulte les statistiques pour s'informer du bon fonctionnement de l'application à travers des courbes visuelles (nombre des services par période, nombre des nouveaux utilisateurs, le type de service le plus demandé...).

2.5 Diagramme d'activités

Le diagramme 2.7 présente de manière visuelle l'enchaînement des fonctionnalités principales de notre logiciel.

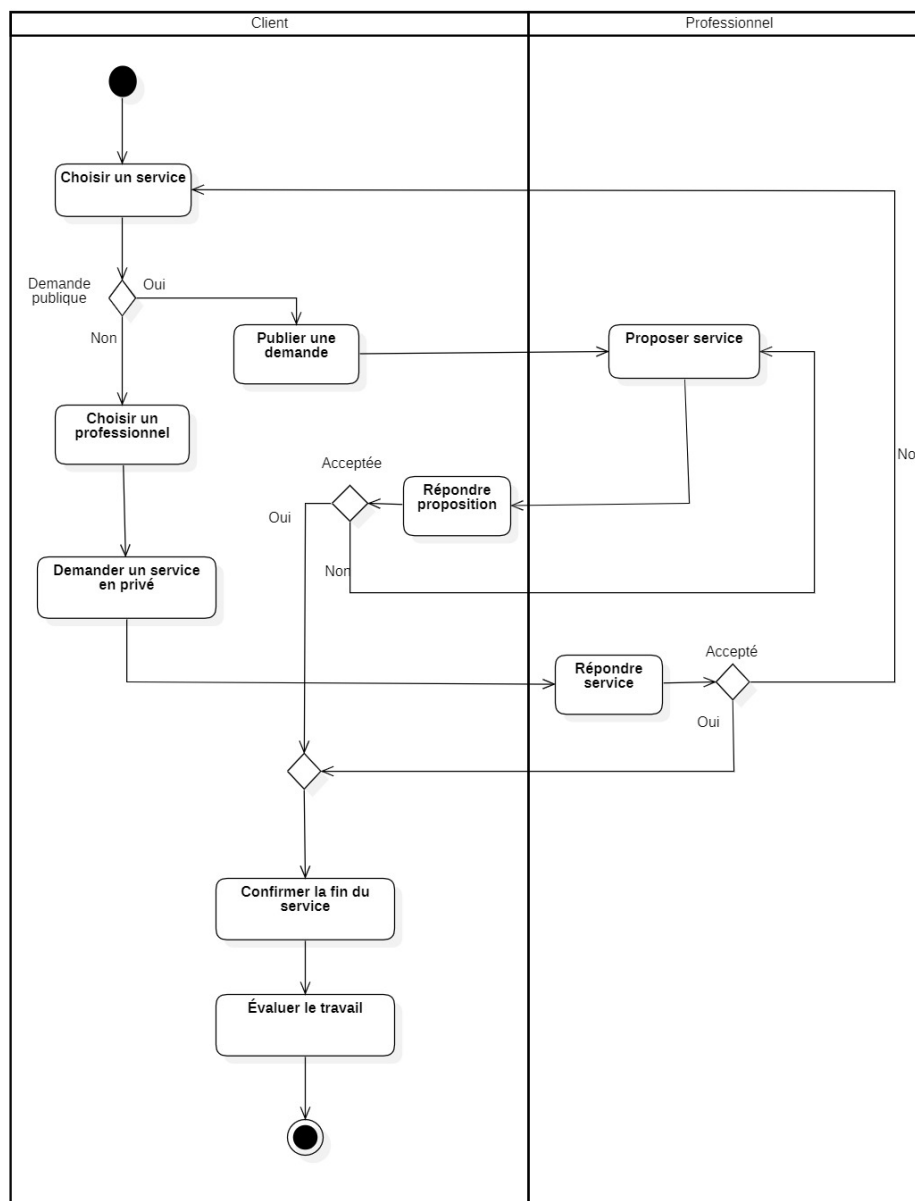


FIGURE 2.7 – Diagramme d'activités partitionné des principales fonctionnalités

Conclusion

En conclusion de ce chapitre, nous avons présenté les besoins fonctionnels et non fonctionnels de notre projet en identifiant les différents acteurs qui interagissent avec notre système. Nous avons utilisé le diagramme de cas d'utilisation pour expliquer les différentes fonctionnalités de l'application et les interactions entre les acteurs et le système. Nous avons également présenté un tableau détaillé pour chaque cas d'utilisation, regroupant toutes les informations nécessaires pour une meilleure compréhension des exigences fonctionnelles de notre projet.

Le prochain chapitre sera consacré à la conception de l'application, en prenant en compte les fonctionnalités décrites dans ce chapitre.

Chapitre 3

Conception

Introduction	23
3.1 Vue statique de l'application	23
3.1.1 Architecture globale du logiciel	23
3.1.2 Définition du patron Clean Architecture	24
3.1.3 Architecture du contrôleur traitements	25
3.1.4 Architecture du contrôleur GUI utilisateur	27
3.1.5 Architecture du contrôleur GUI d'administration	30
3.1.6 Module intelligent de recommandation	31
3.1.7 Module intelligent de vérification	31
3.2 Vue dynamique de l'application	32
3.2.1 Diagrammes de séquences	32
3.2.1.1 Créer un compte professionnel	33
3.2.1.2 Publier une demande de services	34
3.2.1.3 Gérer les demandes publiques des clients	35
3.2.1.4 Demander un service privé d'un professionnel	36
3.2.1.5 Gérer les demandes privées des clients	37
3.2.2 Diagramme d'états-transitions	37
Conclusion	37

Introduction

Après avoir identifié les besoins fonctionnels et non fonctionnels de notre application, nous entamons la partie conception de notre projet dans ce rapport. Nous commençons par décrire la vue statique de l'application en présentant l'architecture générale de notre système et sa décomposition détaillée. Ensuite, nous abordons la vue dynamique en utilisant les diagrammes de séquences, d'activité et d'états-transition.

3.1 Vue statique de l'application

la vue statique de l'application offre une représentation visuelle de la structure et des relations entre les classes et les paquetages de l'application.

3.1.1 Architecture globale du logiciel

Dans cette section, nous définissons la composition structurelle de la solution en indiquant les relations entre ses composants.

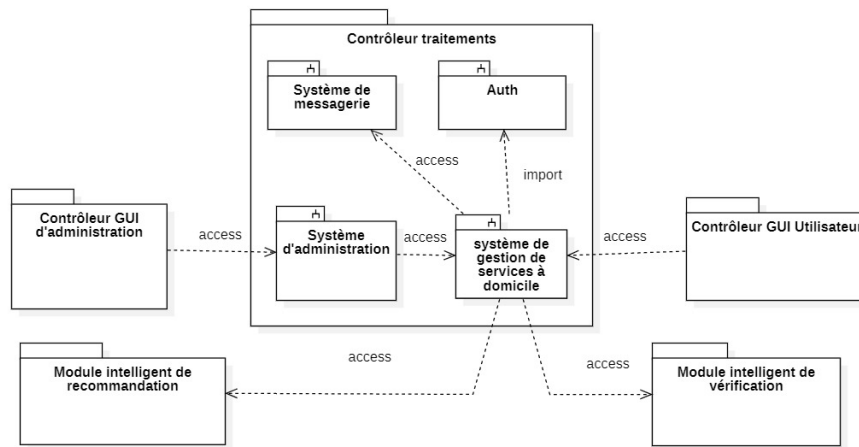


FIGURE 3.1 – Diagramme de paquetages : Vue globale du logiciel

Le schéma de la figure 3.1 illustre l'architecture globale de notre système qui est constituée de cinq paquetages distincts. Chaque paquetage a une fonction précise :

- **Contrôleur traitements** : c'est la partie cœur de notre application.
 - **Système de messagerie** : traite les conversations et les messages entre les utilisateurs (client et professionnel).
 - **Système d'administration** : traite les requêtes d'administration
 - **Système de service à domicile** : responsable de l'authentification des utilisateurs et le traitement de leurs requêtes comme la demande d'un service pour le client, la gestion des demandes des clients pour un professionnel, la gestion du profile ...
- **Contrôleur GUI utilisateur** : représente la partie graphique de notre projet, utilisée par les clients et les professionnels.
 - Visualisation des données afin d'aider l'utilisateur à interagir avec notre système.
 - Gestion des favoris.
 - Contrôle des notifications reçues.
- **Contrôleur GUI d'administration** : c'est une interface d'administration (back-office) pour la visualisation des statistiques et des données d'administration.

- **Module intelligent de recommandation** : analyse les commentaires donnés lors de l'évaluation du travail et attribue un score utile pour recommander des professionnels en classifiant leur liste qui sera affichée au client lors du choix d'un prestataire.
- **Module intelligent de vérification** : vérifie la validité des données personnelles d'un professionnel à partir d'une image de sa carte d'identité à l'aide d'un système d'extraction de texte.

Ces paquetages seront décrits avec détails dans les sous sections suivantes.

3.1.2 Définition du patron Clean Architecture

L'architecture Clean[26] est une méthode de conception logicielle qui repose sur plusieurs principes tels que :

- les principes SOLID[25], qui incluent notamment le principe d'inversion de contrôle, la réduction du couplage entre les composants et le principe d'injection de dépendance.
- Le principe de séparation des règles métiers et des règles de l'application
- L'indépendance du noyau de l'application par rapport aux technologies nécessaires à son fonctionnement.

Grâce à ces principes, les composants de l'application deviennent plus faciles à gérer, à tester, à modifier et à réutiliser.

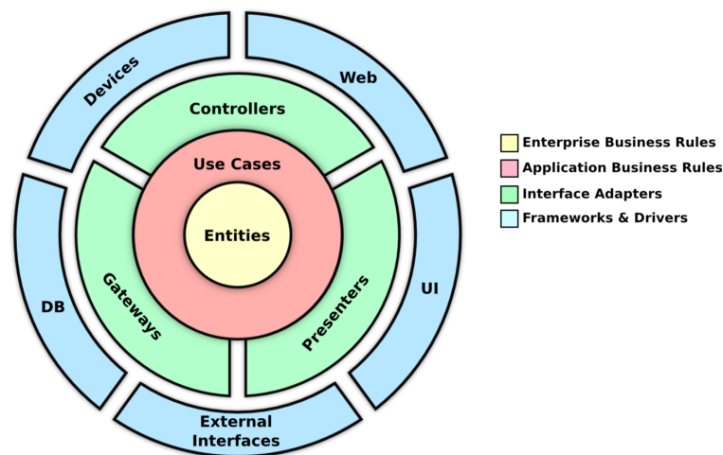


FIGURE 3.2 – Diagramme d'architecture clean [26]

- **Les entités** représentent les objets du domaine de l'application avec leurs méthodes (Business Rules).
- **Les cas d'utilisation** représentent la logique de l'application, c'est-à-dire les règles de gestion spécifiques à une application (Application Rules).
- **Les adaptateurs (Interface Adapters)** représentent l'interface entre les systèmes externes à l'application (frameworks, bases de données, interfaces utilisateur, etc.) et le noyau de l'application (entités + cas d'utilisation). Cette couche permet de réaliser le formatage, la structuration et l'adaptation des données entrantes et sortantes.
- **Les technologies externes** représentent la couche externe à l'application regroupant les technologies nécessaires à son fonctionnement (API, frameworks, bases de données, interfaces utilisateur, réseaux, drivers, appareils...).

[22, p. 27 - 29]

Nous expliquerons dans les sections 3.1.3 et 3.1.4 comment nous avons appliqué ce patron dans notre projet.

3.1.3 Architecture du contrôleur traitements

Il s'agit d'une partie du backend qui sert à traiter les requêtes envoyées par le contrôleur GUI et la partie d'administration. Cette partie est responsable de la gestion de la logique métier de l'application et du traitement des données et des requêtes de l'utilisateur. Elle assure une communication fluide et efficace entre les différents composants de l'application.

Application du patron d'architecture clean

Dans le paquetage "contrôleur traitements", nous avons appliqué l'architecture clean. Pour la communication entre les couches, nous appliquons le principe d'inversion de contrôle "IoC" en utilisant un framework externe à l'application qui crée les composants de l'application et injecte les dépendances entre eux selon le principe d'injection de dépendances.

La figure 3.3 montre un exemple d'application de l'architecture clean pour le sous paquetage "Auth".

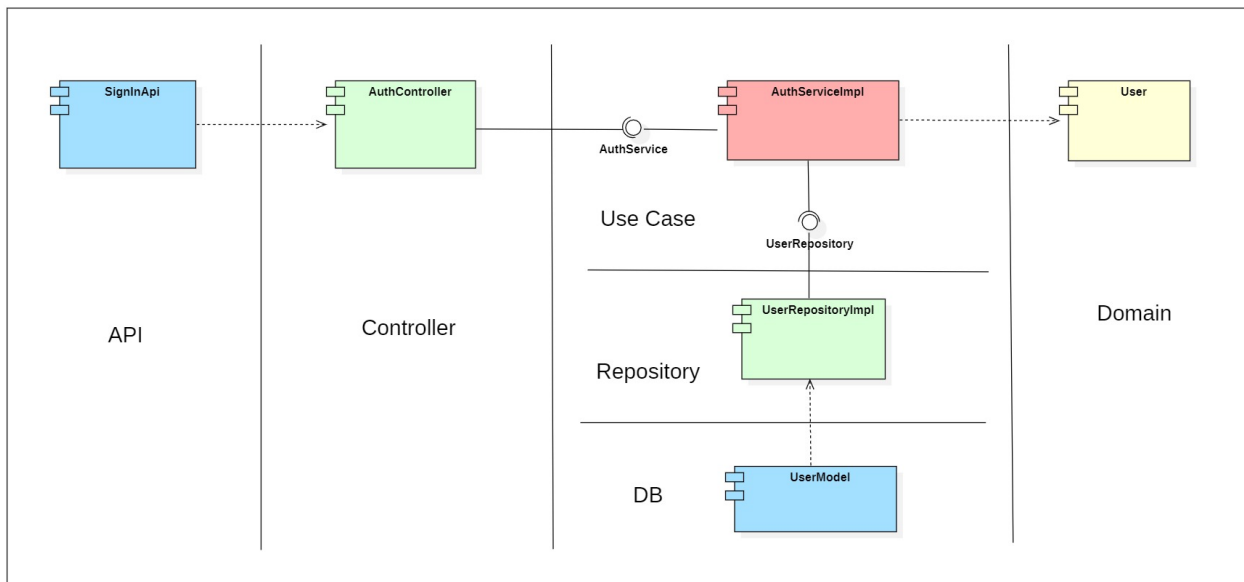


FIGURE 3.3 – Diagramme de composants détaillé du sous-paquetage : Auth

Diagramme de classes de la couche Entités

La figure 3.4 illustre le diagramme de classes de la couche centrale du patron clean, la couche **Entités** qui regroupe les classes représentant les principaux objets de notre applications.

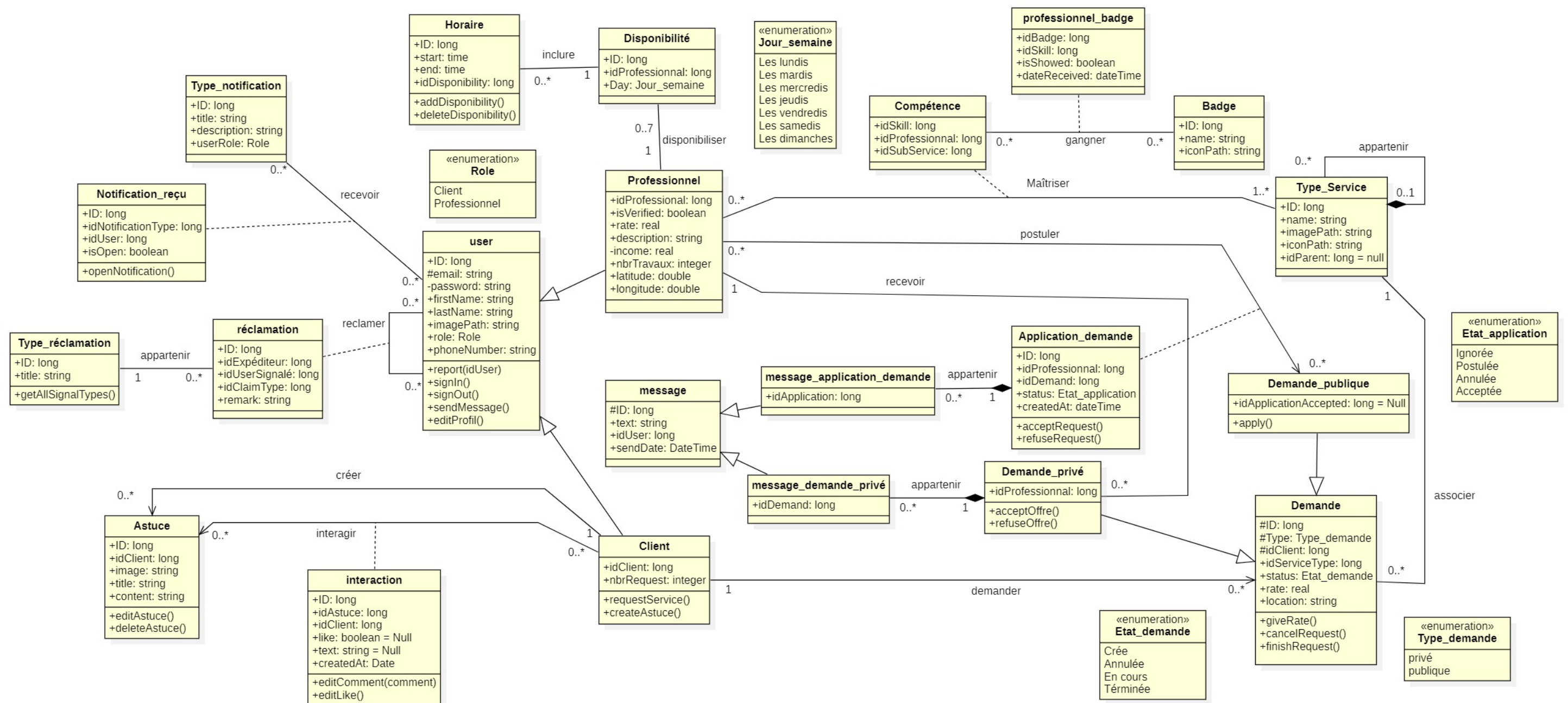


Figure 3.4 - Diagramme de classes de la couche Entités

Description des classes du diagramme illustré par la figure 3.4 :

- **User** c'est l'entité principale de notre système, qui peut être soit un client soit un professionnel.
 - **Client** : le demandeur de service à domicile.
 - **Professionnel** : le prestataire qui offre un service.
- **Réclamation** : chaque utilisateur peut envoyer une réclamation à l'administration concernant un autre utilisateur.
- **Notification reçu** : les utilisateurs de notre plateforme reçoivent des notifications concernant leurs demandes et les nouvelles publications.
- **Badge** : Pour un milieu compétitif, les professionnels peuvent gagner des badges d'excellence en fonction de leurs activités.
- **Disponibilité** : les professionnels ajoutent leurs disponibilités en précisant leurs horaires de travail pour chaque jour.
- **Compétence** : les compétences de travail que le professionnel peut réaliser
- **Type service** : les services de dépannage à domicile disponibles dans la plateforme
- **Astuce** : notre système permet également l'échange d'astuces de bricolage entre les clients, chaque client peut ajouter, modifier ou supprimer ses astuces et d'interagir avec des commentaires.
- **Demande** les demandes des clients, qui peuvent être soit des demandes publiques (pour lesquelles les clients peuvent proposer des services) ou des demandes privées (envoyées à un professionnel spécifique).
- **Application demande** : les propositions de services envoyées par les professionnels pour une demande publique.

3.1.4 Architecture du contrôleur GUI utilisateur

Il s'agit d'un module frontend mobile destiné à l'utilisateur (client ou professionnel). Ce module représente une interface de communication avec notre contrôleur de traitement backend. Sa fonction principale est d'afficher les données et les fonctionnalités pertinentes pour l'utilisateur, en communiquant avec le backend pour récupérer et envoyer les données nécessaires. Il peut également contenir de la logique de présentation pour rendre l'interface utilisateur conviviale et attrayante.

Diagramme de paquetages détaillé

Le contrôleur GUI utilisateur dispose d'une architecture modulaire réalisant des fonctionnalités distinctes, ce qui garantit leur réutilisabilité. Le diagramme suivant permet d'illustrer cette architecture.

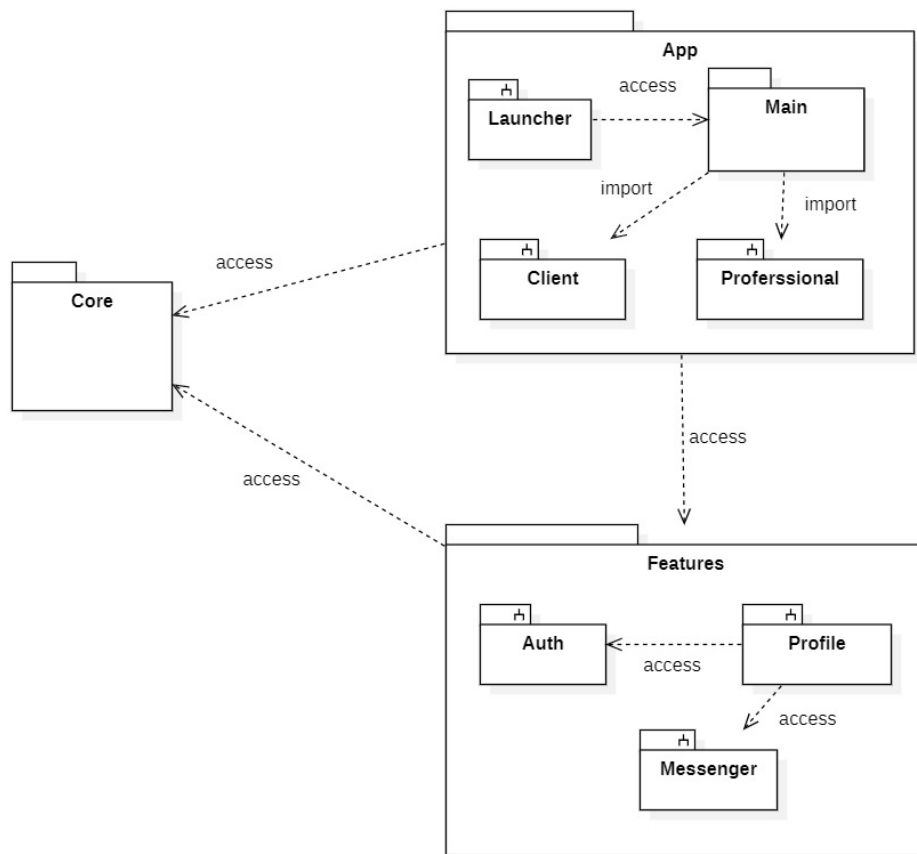


FIGURE 3.5 – Diagramme de paquetages détaillé : Contrôleur GUI utilisateur

- **App** : joue le rôle principal de lancement de notre application en faisant appel à d'autres modules des autres sous-paquetages .
- **Core** : regroupe les fonctionnalités communes à plusieurs modules du paquetage "Contrôleur GUI utilisateur" afin de minimiser la redondance du code et favoriser le principe de réutilisation.
- **features** : ce sous-paquetage est organisé en fonctionnalités (ou "features") telles que la connexion (le module "Auth"), la messagerie (le module "Messenger"), la gestion profil (le module "Profile"). Le module "Auth" du sous-paquetage "features" du paquetage "Contrôleur GUI utilisateur" lance à son tour le module "Auth" du paquetage "Contrôleur traitement".

Application du patron d'architecture clean

Chaque sous-package dans la figure 3.5 est conçu selon l'architecture clean. La figure 3.6 illustre le diagramme de classes du sous-paquetage "Core" en montrant l'application de l'architecture clean : la couleur de chaque classe représente son appartenance à l'une des couches de cette architecture (voir figure 3.2) .

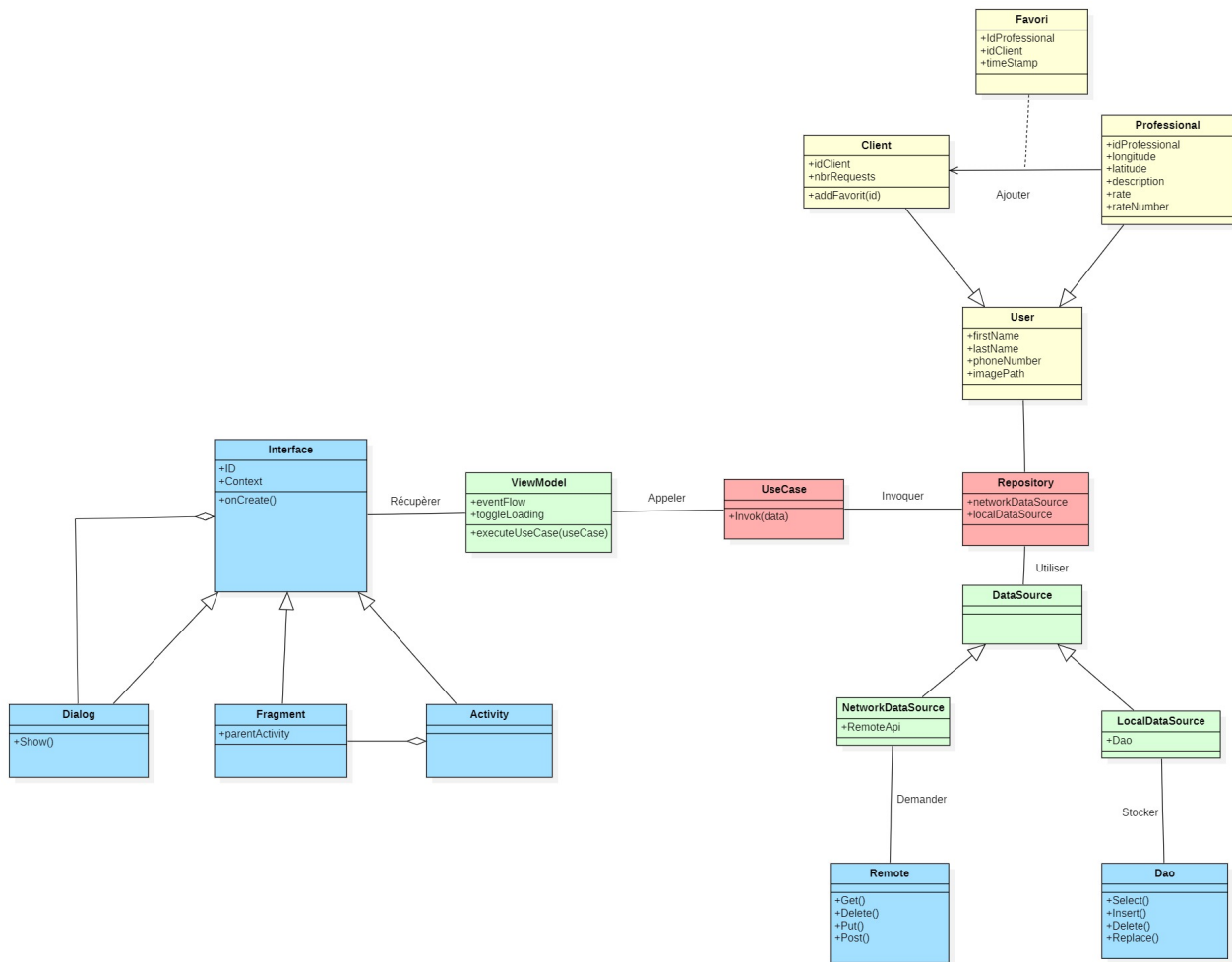


FIGURE 3.6 – Diagramme de classe du sous-paquetage : Core

- **Interface** : la classe représentative des éléments visuels et interactifs qui facilitent l'interaction de l'utilisateur avec notre application.
- **ViewModel** : Conserve les données reçues afin de les récupérer ultérieurement via la vue. Cette classe fait partie du patron d'architecture MVVM, défini à la fin de cette section, également appliqué dans ce sous-paquetage Core.
- **UseCase** : Pour la communication entre la couche presenter "ViewModel" et la couche use case "Repository".
- **Repository** : Représente les objets qui encapsulent la logique de l'application et assurent la gestion des données permettent à notre application de fonctionner de manière hors ligne. Ces objets synchronisent les données localement en l'absence de connexion et les utilisent en cas de perte de connexion.
- **Favori** : c'est la liste des professionnels en favori pour les clients.
- **Data source** : Prépare les données, pour les rendre utilisables dans notre application.
 - **Remote** : Représentent les objets responsables de la communication avec des sources de données distantes et permettent de récupérer les données à partir de ces sources.
 - **Dao** : Représente les objets utilisés pour manipuler les données stockées localement.

La figure 3.7 représente une instantiation du digramme de classes de la figure 3.5, où les instances des classes sont modélisés par des composants logiques.

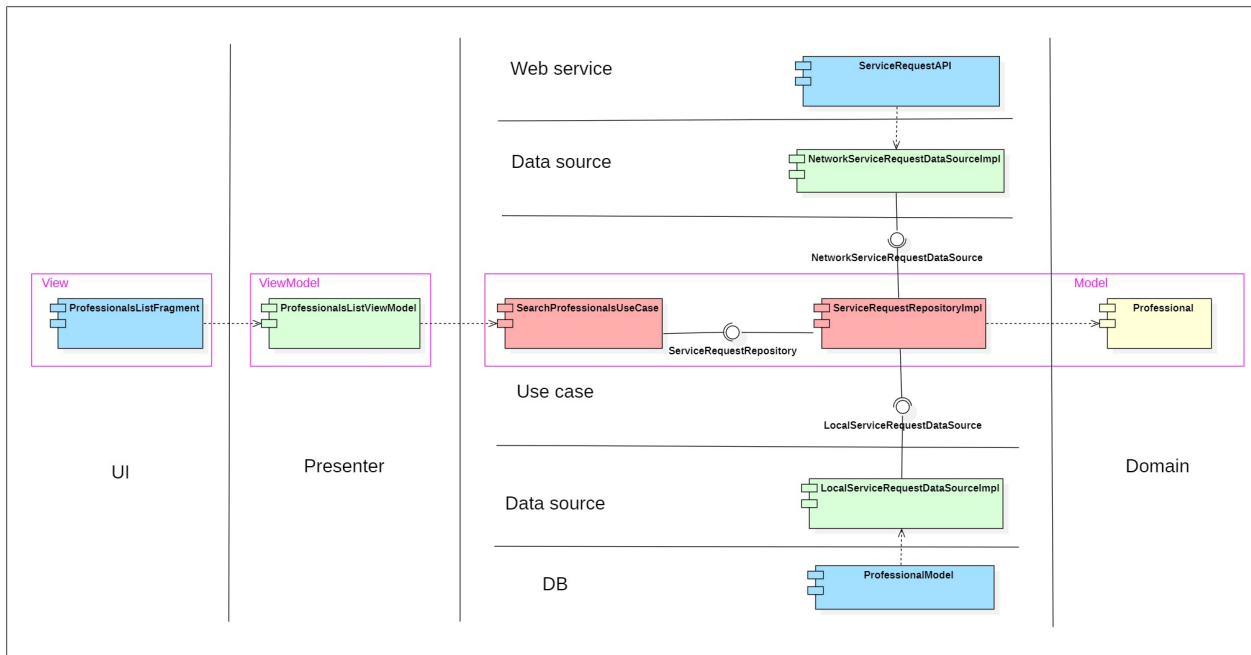


FIGURE 3.7 – Diagramme de composants détaillé d'une instance du Core

Nous avons appliqué aussi le patron d'architecture MVVM comme indiqué dans le figure 3.7 dans ce sous-paquetage "Core". MVVM signifie Model-View-ViewModel, qui se compose de trois parties :

- **View** : correspond à l'interface utilisateur affichée à l'utilisateur.
- **Model** : représente les objets métier qui encapsulent les données et les règles de gestion de l'application.
- **ViewModel** : récupère les données à partir du modèle et les stocke dans des binding attributes consultés ensuite par la vue pour les récupérer.

3.1.5 Architecture du contrôleur GUI d'administration

Le module d'administration dans notre système est conçu pour un seul administrateur, implémenté en tant que singleton. Ce module fournit une interface utilisateur conviviale sous la forme d'un dashboard web pour permettre à l'administrateur d'afficher des statistiques visuelles sous la forme des chartes graphiques liées aux services fournis par notre système (nombre des services par période, nombre des nouveaux utilisateurs, le type de service le plus demandé...). Ce module permet aussi de bloquer ou débloquer les comptes utilisateurs et de gérer les types de services disponibles.

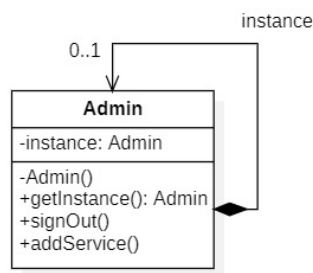


FIGURE 3.8 – Diagramme de classes du paquetage : Contrôleur GUI d'administration

La figure 3.8 illustre l'implémentation du patron de conception Singleton[24] dans notre paquetage

d'administration. Ce patron se compose principalement d'une classe qui doit avoir une seule instance pendant l'exécution. Ce type de classe crée sa propre instance une fois seulement et la fournit à ses classes clientes en utilisant la méthode `getInstance()`.

3.1.6 Module intelligent de recommandation

Ce module est développé indépendamment de notre application et peut être réutilisé dans d'autres projets. Il a pour rôle de calculer un score en utilisant le nombre d'étoiles et un commentaire. En effet, il s'agit d'un algorithme de recommandation (Algorithm 1) qui prend en entrée le nombre d'étoiles sélectionnées et le commentaire écrit par le client. Les mots qui compose ce commentaires seront indexés selon un dictionnaire de mots prédéfini (Instruction 1 : Préparer C). Ensuite, chaque mot sera classifié selon deux classes (positif : 1, négatif : 0) et un pourcentage P sera prédit représentant la qualité du commentaire (Instruction 2 : Prédire P). La classification des mots du commentaire et la prédiction du pourcentage de qualité de ces mots sont appliquées en utilisant des techniques de Machine Learning[27]. Après la prédiction, un score est calculé selon le pourcentage retourné et l'évaluation donnée par le client.

Algorithm 1 Algorithme de recommandation

Entrée : Nombre des étoiles N, Commentaire C

Début

1. **Préparer** C pour la prédiction
2. **Prédire** P, en passant C à notre modèle
3. **Si** P est compris entre 0 et 0.2, alors R = 1
 Sinon, si P est compris entre 0.21 et 0.4, alors R = 2
 Sinon, si P est compris entre 0.41 et 0.6, alors R = 3
 Sinon, si P est compris entre 0.61 et 8, alors R = 4
 Sinon R = 5
4. **Retourner** (R + N) / 2

Fin

3.1.7 Module intelligent de vérification

Pour la vérification automatique des données personnelles de nos professionnels, nous avons développé l'algorithme 2. Ce mécanisme prend en entrée le chemin de l'image de la carte d'identité ainsi que les données à vérifier. Ensuite un prétraitement de filtrage et de transformation en niveaux de gris est appliqué sur l'image. Puis, une extraction du texte existant dans l'image est appliquée en utilisant une technique d'OCR (Optical Character Recognition)[14]. Enfin, cet algorithme retourne "vrai" s'il existe un matching entre les données extraites et les données saisies.

Algorithm 2 Algorithme de vérification

Entrée : Chemin de l'image PATH, nom N, prénom P, numéro cin C

Début

1. **Obtenir** l'image I, à partir du chemin PATH.
2. **Préparer** I, en exécution des filtres et de transformation en niveaux de gris.
3. **Extraire** nom N1, prénom P1, numéro cin C1, de l'image du carte d'identité I
4. **Si** N1 == N et P1 == P et C1 == C alors
 retourner Vrais
 Sinon, retourner Faux

Fin

3.2 Vue dynamique de l'application

La conception dynamique de l'application permet de décrire le fonctionnement interne de notre application en utilisant différents diagrammes UML. Ces diagrammes nous aideront à représenter la séquence d'actions et le flux de données circulant à travers les différentes parties de notre application. Nous allons notamment utiliser des diagrammes de séquences et un diagramme d'états transition pour modéliser et visualiser les interactions circulant à travers les différents composants de notre application. Ces diagrammes nous permettront de mieux comprendre le comportement de l'application.

3.2.1 Diagrammes de séquences

Le diagramme de séquence est utilisé pour modéliser l'interaction entre les acteurs principaux et les composants de l'application. Dans notre projet, nous avons choisi de représenter la fonctionnalité de création d'un compte client avec le diagramme de séquences détaillées ainsi que deux autres fonctionnalités principales : la demande d'un service et la gestion des demandes des clients (publiques et privées).

3.2.1.1 Créer un compte professionnel

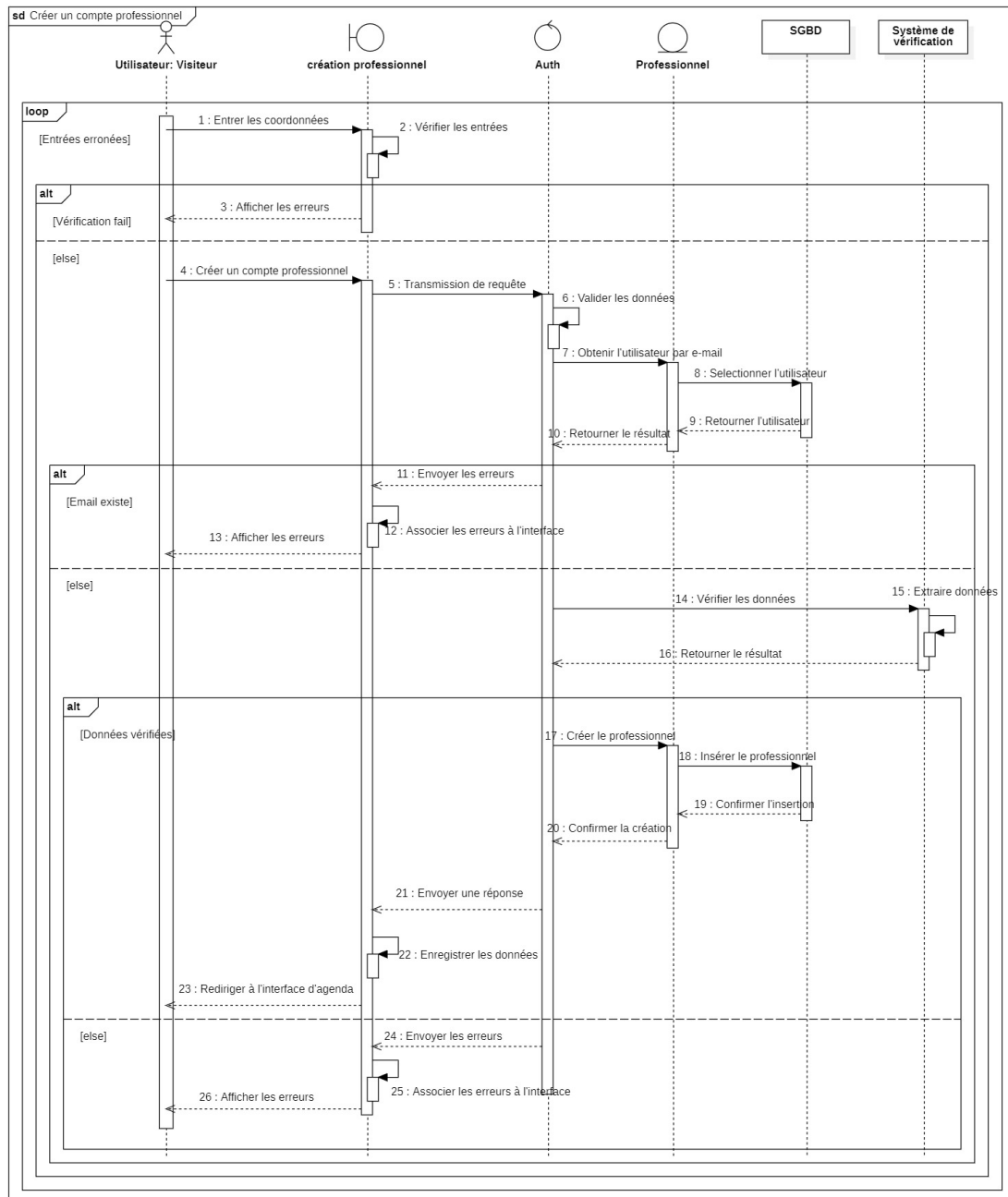


FIGURE 3.9 – Diagramme de séquences détaillées modélisant la création d'un compte professionnel

Le diagramme 3.9 illustre le processus de création d'un compte professionnel par le visiteur, ainsi que la communication entre les différents composants de notre système pour assurer cette création.

3.2.1.2 Publier une demande de services

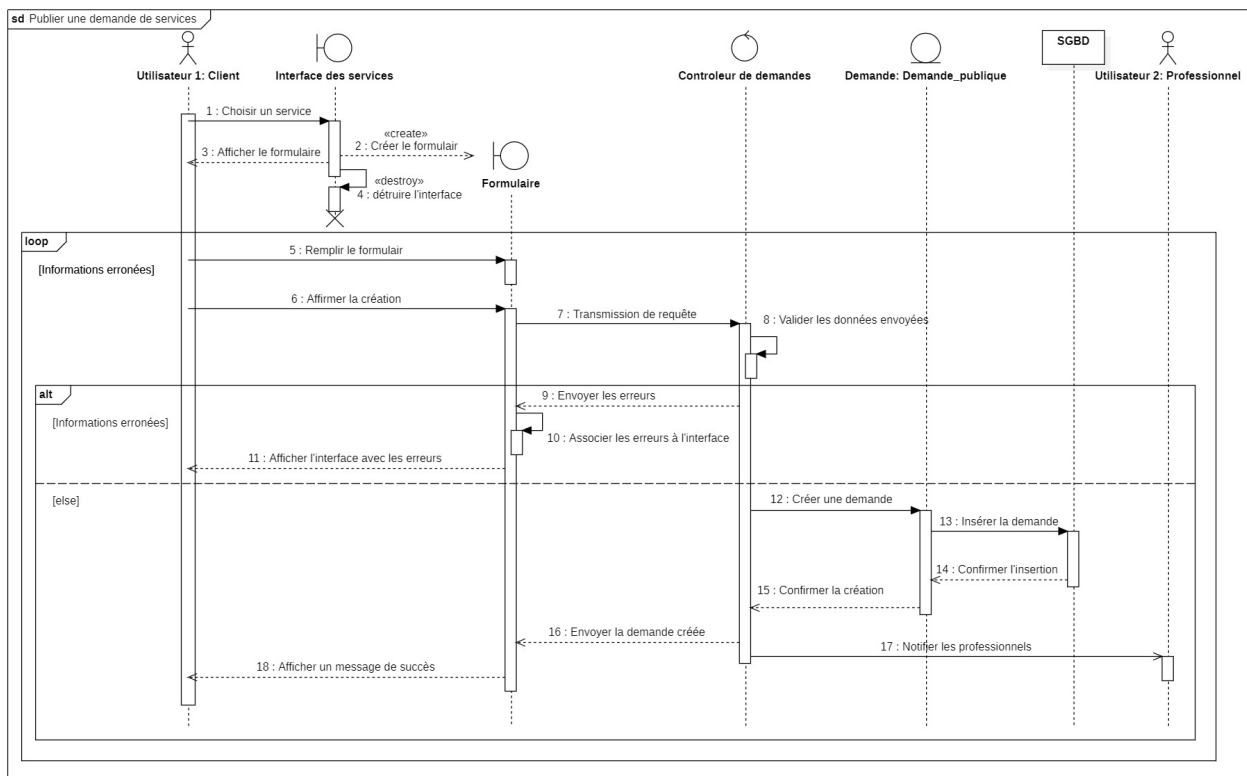


FIGURE 3.10 – Diagramme de séquences détaillées modélisant la demande publique d'un service

Le diagramme 3.10 illustre le processus de demande publique de service à domicile, avec toutes les étapes de traitement nécessaires.

3.2.1.3 Gérer les demandes publiques des clients

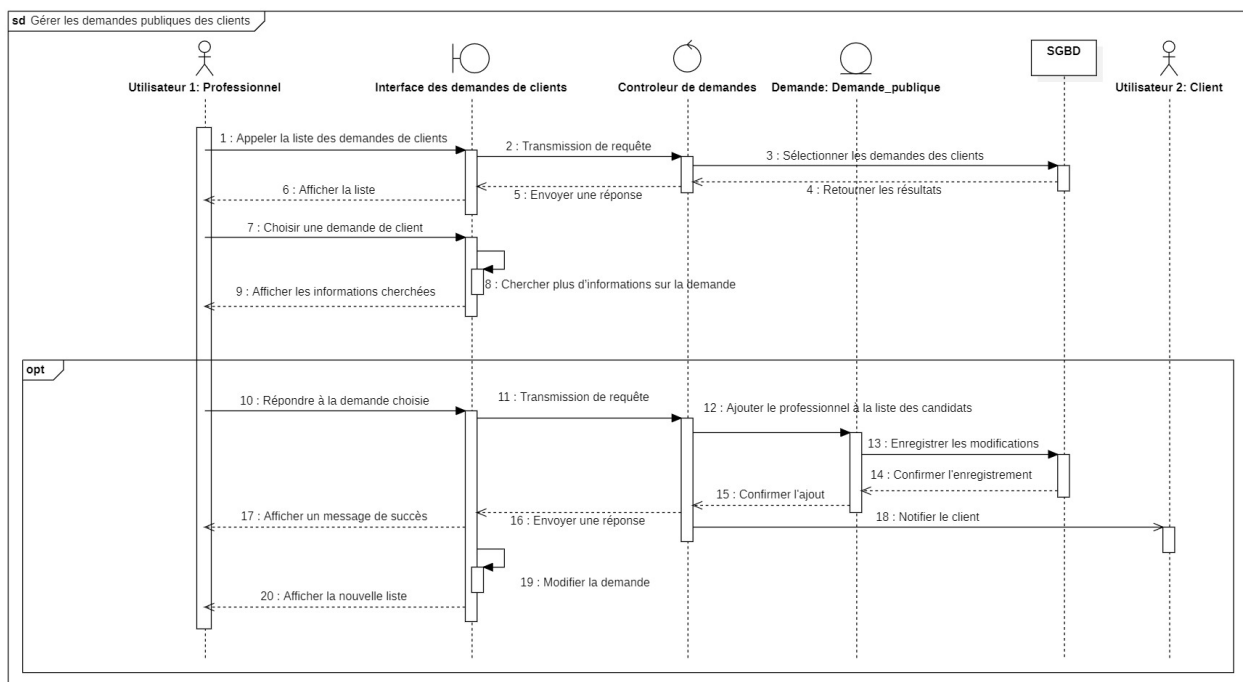


FIGURE 3.11 – Diagramme de séquences détaillées modélisant la gestion des demandes publiques des clients

Le diagramme 3.11 représente le processus de traitement pour gérer les demandes publiques des clients. Les demandes des clients sont publiées dans un espace de publication accessible par tous les professionnels. Le diagramme montre comment le professionnel peut consulter la liste des demandes disponibles, et répondre à une demande en proposant des services.

3.2.1.4 Demander un service privé d'un professionnel

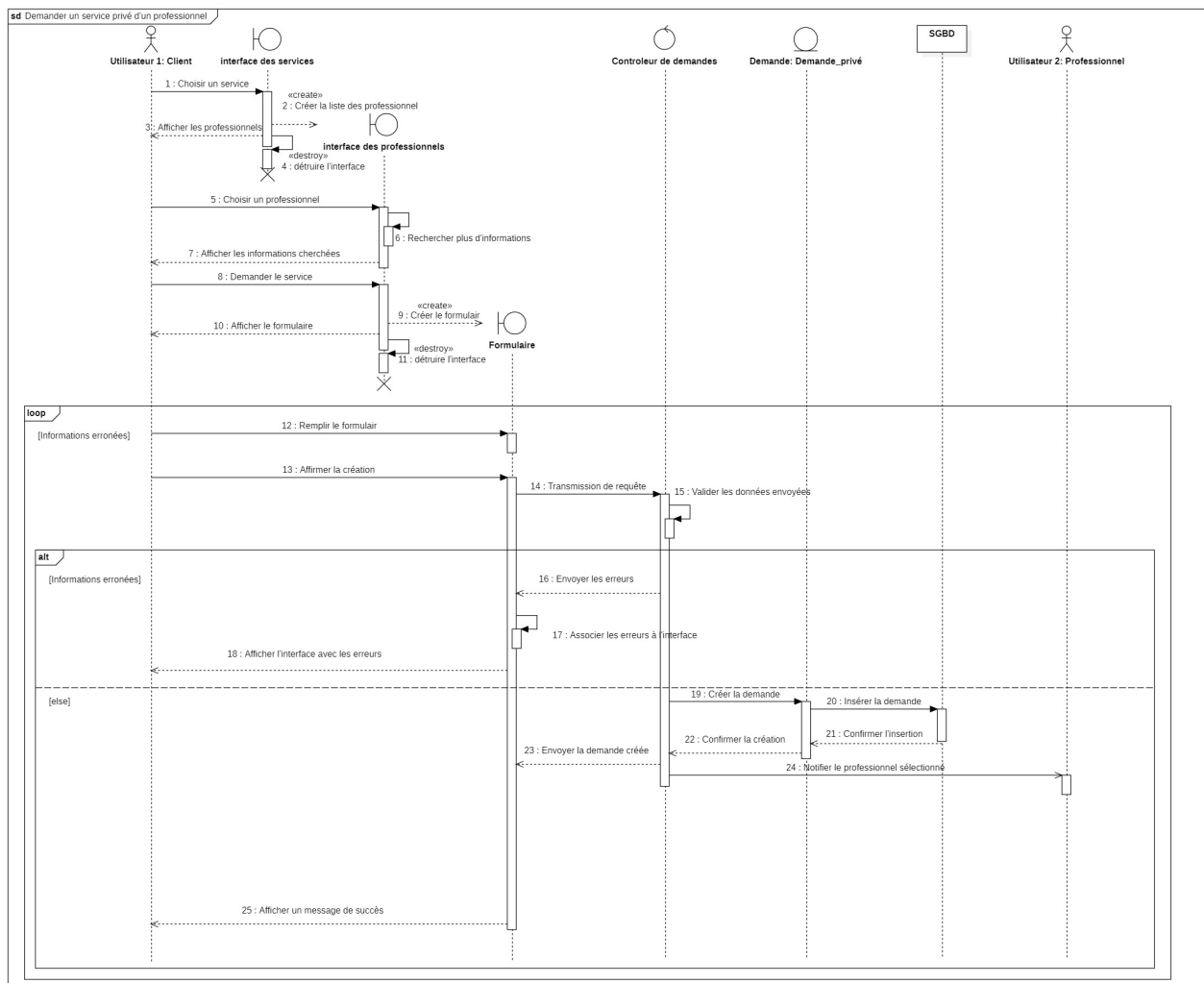


FIGURE 3.12 – Diagramme de séquences modélisant la demande d'un service d'un professionnel spécifique

Le diagramme 3.12 illustre le processus de demande de service à domicile par le client en indiquant le professionnel désiré.

3.2.1.5 Gérer les demandes privées des clients

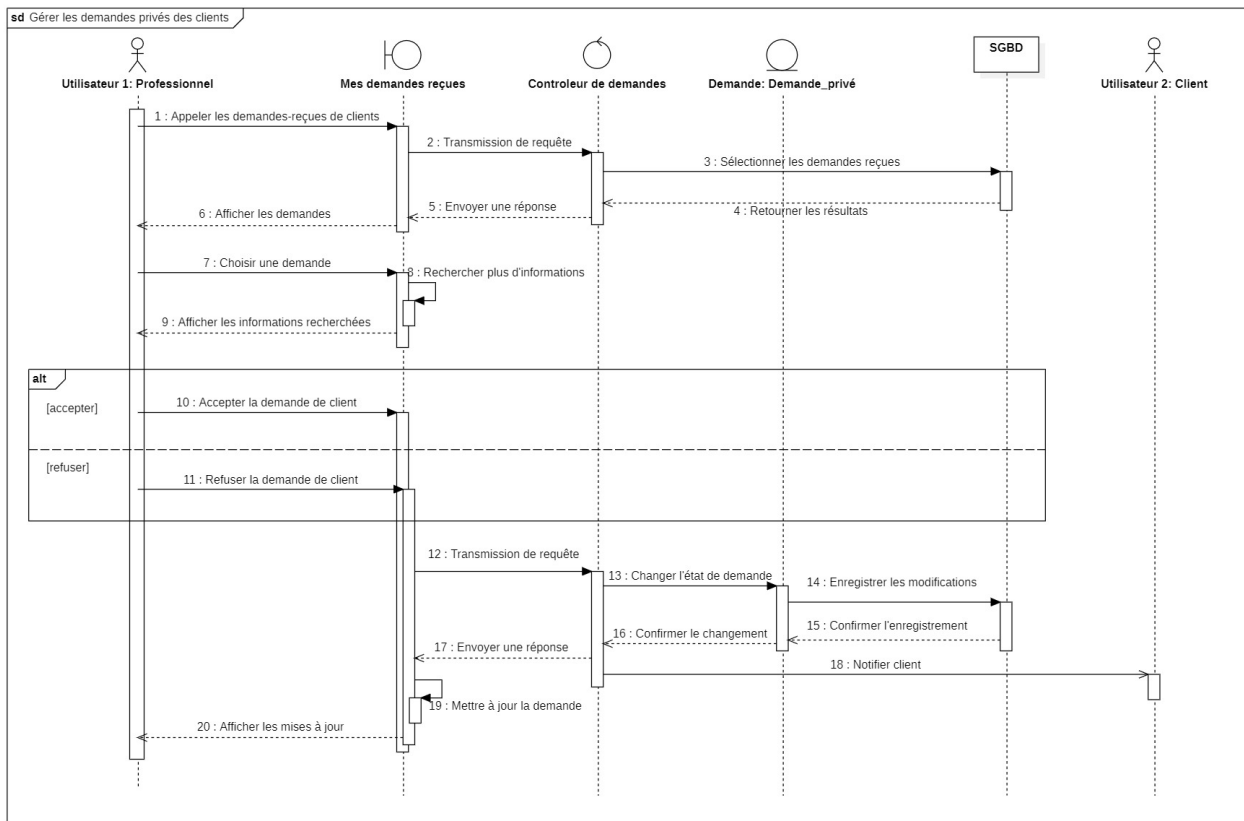


FIGURE 3.13 – Diagramme de séquences modélisant la gestion de mes demandes reçues

Le diagramme 3.13 montre le processus de traitement pour que le professionnel gère ses demandes reçues. Le professionnel peut alors visualiser les détails de la demande et y répondre en conséquence

3.2.2 Diagramme d'états-transitions

Pour décrire le cycle de vie d'une demande dans les cas précédents, le diagramme d'états-transitions décrit dans la figure 3.14 explique les changements.

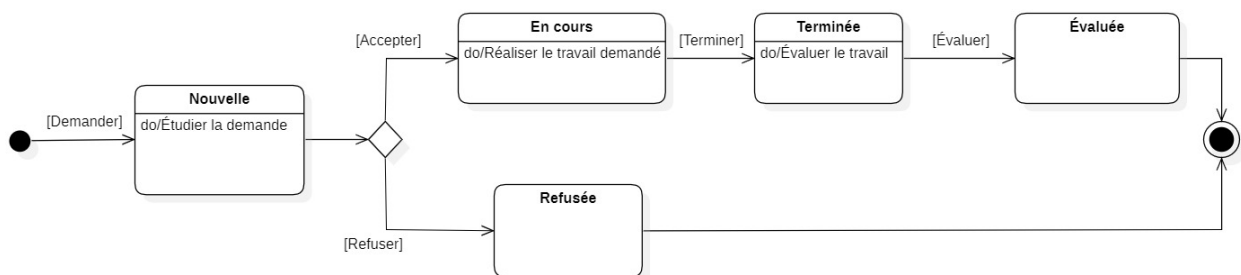


FIGURE 3.14 – Diagramme d'états-transitions pour une demande de services à domicile

Conclusion

Au terme de ce chapitre, nous avons présenté l'architecture de notre logiciel ainsi que les différents diagrammes de conception statique et dynamique que nous avons élaborés. Nous avons ainsi pu décrire

de manière détaillée le comportement de notre application ainsi que les interactions entre ses différents composants. Cette phase de conception nous a permis d'achever la phase de modélisation et de poser les bases solides pour la phase de développement.

Chapitre 4

Réalisation

Introduction	40
4.1 Technologies	40
4.1.1 Outils logiciels	40
4.1.2 Langage de développement	41
4.1.3 API et bibliothèques utilisée	41
4.2 Outils d'implémentation	41
4.3 Architecture physique	42
4.4 Interfaces de l'application	43
4.4.1 Lancement de l'application	43
4.4.2 Authentification	43
4.4.3 Gestion profile	44
4.4.4 Demander un service	46
4.4.5 Gestion des demandes des clients	46
4.4.6 Messagerie	47
4.4.7 Évaluation	48
4.4.8 Administration	48
Conclusion	50

Introduction

Dans le cadre de la réalisation de notre application, nous avons utilisé une variété d'outils. Ce chapitre est dédié à la description de ces outils ainsi qu'à la présentation de notre environnement de travail. De plus, nous présenterons les différentes fonctionnalités de notre application à travers des captures d'écran, permettant ainsi d'illustrer le travail accompli.

4.1 Technologies

Dans cette section, nous procédons à la définition des technologies qui constituent l'environnement de développement de notre logiciel.



FIGURE 4.1 – Technologies utilisées

- **Android native** nous a permis d'écrire du code natif, généralement en utilisant le kit de développement logiciel (SDK). Cela nous donne la possibilité d'accéder à des fonctionnalités du système d'exploitation Android qui ne sont pas disponibles ou limitées dans le cadre du développement d'applications Android standard pour le développement du paquetage contrôleur GUI utilisateur (voir section 3.1.4).
- **Node.js avec Express** nous ont aidé à créer le paquetage contrôleur traitements (voir section 3.1.3), en profitant de la puissance de JavaScript et des fonctionnalités supplémentaires offertes par le framework Express.
- **React**[15] est utilisé pour le développement du contrôleur GUI d'administration. c'est une bibliothèque JavaScript utilisée pour la construction d'interfaces utilisateur interactives et réactives.

4.1.1 Outils logiciels

Au cours de l'implémentation de notre projet, nous avons utilisé les logiciels suivants :

- **Figma**[6] est un outil de conception d'interfaces utilisateur. Nous avons utilisé Figma dans notre projet, notamment pour la conception et le prototypage de l'interface utilisateur de notre application.
- **Android studio**[2] est un environnement de développement intégré (EDI) développé par Google, spécialement conçu pour le développement d'applications Android. Nous avons également utilisé cet outil pour la partie frontend de notre projet, afin de concevoir l'interface utilisateur et les fonctionnalités de notre application Android.
- **Visual Studio Code**[21] est un éditeur de texte développé par Microsoft, spécialement conçu pour le développement. Nous avons utilisé cet outil pour la partie backend de notre projet.
- **GitLab**[9] est une plateforme web de gestion de développement de logiciels basée sur Git. Elle offre un ensemble complet d'outils pour la gestion des versions et la gestion du code source.
- **Laragon**[13] est un environnement de développement rapide et puissant que nous avons utilisé pour la base de données MySQL.

4.1.2 Langage de développement

Pour le développement de notre application, nous avons utilisé les langages suivants :

- **Typescript**[20] est un langage de développement maintenu par Microsoft. Il ajoute des fonctionnalités de typage statique à la syntaxe de JavaScript, ce qui permet de détecter les erreurs potentielles dès la phase de développement. Nous avons choisi d'utiliser TypeScript dans notre backend pour améliorer la robustesse et la maintenabilité du code.
- **Kotlin**[12] est un langage de programmation moderne, concis et sécurisé, développé par JetBrains. Il est compatible avec la machine virtuelle Java (JVM) et peut être utilisé pour le développement d'applications Android, ainsi que pour d'autres plateformes.
- **XML** nous offre la possibilité de créer des formats d'information et de les partager via les réseaux. Nous avons également utilisé ce langage pour la création des interfaces Android.

4.1.3 API et bibliothèques utilisée

- **SocketIO**[18] est une bibliothèque JavaScript qui facilite la communication en temps réel entre le client et le serveur.
- **Swagger**[19] est un ensemble d'outils open source permettant de concevoir, créer, documenter et tester des API RESTful de manière efficace.
- **Inversify**[11] est un conteneur d'injection de dépendances (IoC) pour JavaScript/TypeScript. Il nous permet de gérer les dépendances entre les différentes composantes de notre serveur.
- **Firebase Cloud Messaging (FCM)**[7] est un service de messagerie cloud proposé par Firebase. FCM nous permet d'envoyer des notifications push et des messages aux utilisateurs de notre application.
- **Google Maps API**[10] nous avons utilisé l'API Google Maps pour la localisation des utilisateurs et des demandes services.
- **Flow**[8] est un type de structure de données qui peut émettre plusieurs valeurs de manière séquentielle dans les coroutines Kotlin.
- **Coroutines Kotlin**[4] est utilisé pour exécuter un bloc de code de manière asynchrone sous forme de threads légers.
- **Retrofit**[16] est un client REST pour Android et java qui consomme les web services.
- **Hilt**[5] pour l'injection de dépendances android.
- **Room**[17] est une base de données android relationnelle.
- **Coil**[3] est utilisé dans notre application pour afficher les images distantes stockées dans notre serveur.

4.2 Outils d'implémentation

Pour le développement de cette application nous avons disposé du matériel suivant :

- Un ordinateur personnel
 - Marque : MSI.
 - Système d'exploitation : Windows 11.
 - Processeur : Intel CORE i7.
 - GPU : Nvidia GTX 1660.
 - RAM : 16 Go.
 - Stockage : 1 To HDD, 480 Go SSD.
- Smartphone virtuelle en utilisant windows 11 subsystem

- Marque : Pixel 5.
- Système d'exploitation : Android 13.
- Processeur : Snapdragon 765G 5G.
- RAM : 6 Go.
- Stockage : 256 Go.
- Smartphone virtuelle en utilisant Android studio
 - Marque : Pixel 4.
 - Système d'exploitation : Android 11.
 - Processeur : Qualcomm Snapdragon 855.
 - RAM : 2 Go.
 - Stockage : 8 Go.
- Smartphone physique
 - Marque : Samsung s6.
 - Système d'exploitation : Android 8.
 - Processeur : Exynos 7 Octa 7420.
 - RAM : 3 Go.
 - Stockage : 32 Go.

4.3 Architecture physique

Le diagramme de déploiement offre une représentation de l'agencement matériel et logiciel de l'application. En effet, ce diagramme combine l'architecture logique et l'architecture physique du logiciel.

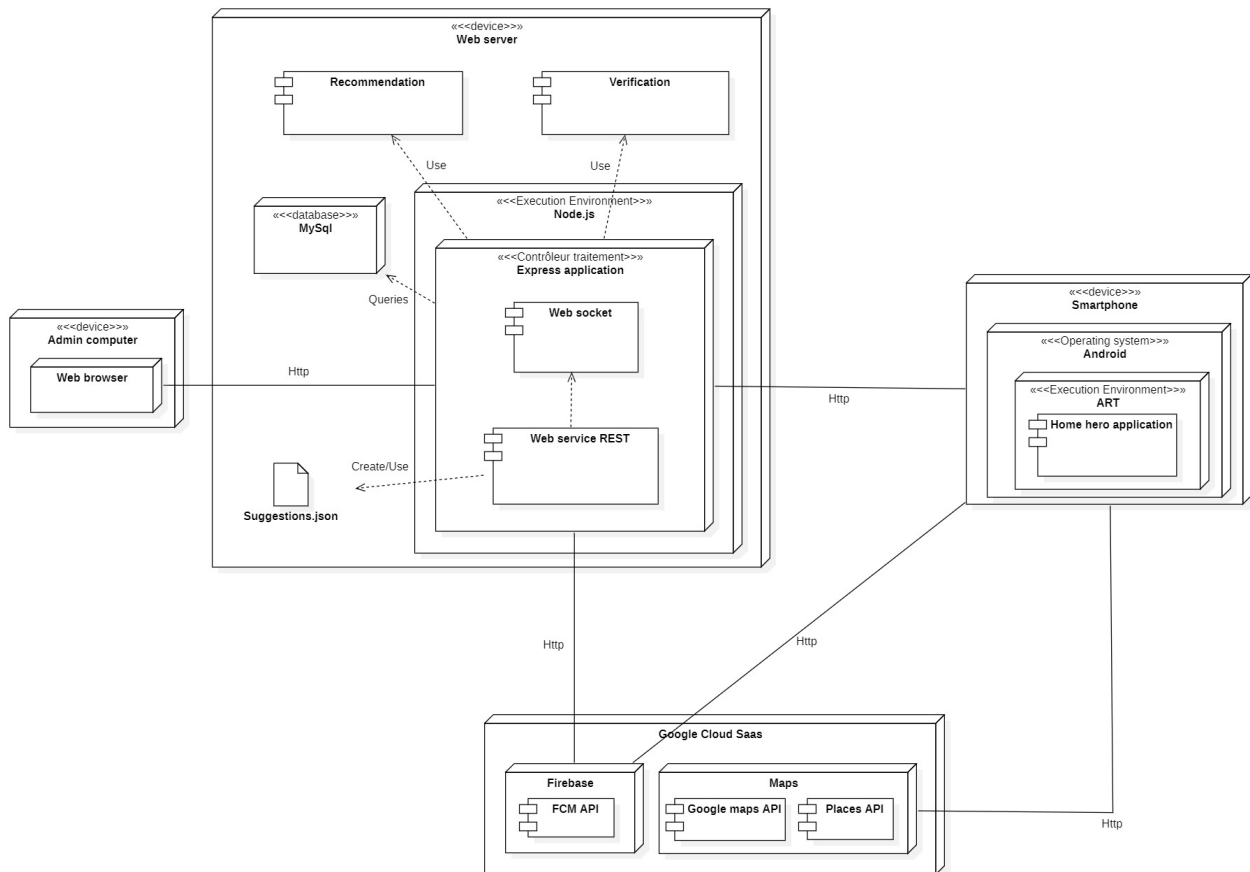


FIGURE 4.2 – Diagramme de déploiement de notre application

Le diagramme de déploiement de notre application dans la figure 4.2 se compose de 4 noeuds :

- **Smartphone** est un appareil qui intègre le système d'exploitation mobile Android, sur lequel notre application nécessite l'installation et l'exécution du fichier .apk à l'aide de l'Android Runtime (ART)[1].
- **Serveur web** est responsable de l'hébergement de l'application "Express.js" qui fournit les API et les Web Sockets utilisés par notre plateforme. Il joue le rôle de backend de notre plateforme.
 - **Suggestions.json** est généré afin d'optimiser le processus de recommandation des professionnels. Notre application effectue quotidiennement le calcul de l'évaluation des services sur une période de sept jours et met à jour le fichier pour permettre aux clients d'y accéder directement, sans avoir besoin de recalculer à chaque fois.
- **Cloud SAAS** est un modèle de distribution permettant l'accès à des services via le cloud pour les utiliser. Parmi ces services, nous avons utilisé Firebase Cloud Messaging (FCM), Google Maps API et Places API.
- **Admin computer** est l'appareil d'accès utilisé par l'administrateur avec un navigateur.

4.4 Interfaces de l'application

Dans cette section, nous présentons les interfaces graphiques de notre application qui indiquent les différentes fonctionnalités.

4.4.1 Lancement de l'application

La figure (4.3a) représente l'écran de démarrage (splash screen) qui indique le lancement de l'application. Pendant ce processus de démarrage, des vérifications sont effectuées afin de déterminer quelle interface à afficher. Ensuite, la figure (4.3b) illustre l'interface d'intégration de nouveaux utilisateurs qui n'apparaît qu'à la première ouverture de l'application. Cette interface fournit des informations sur les fonctionnalités de la plateforme.

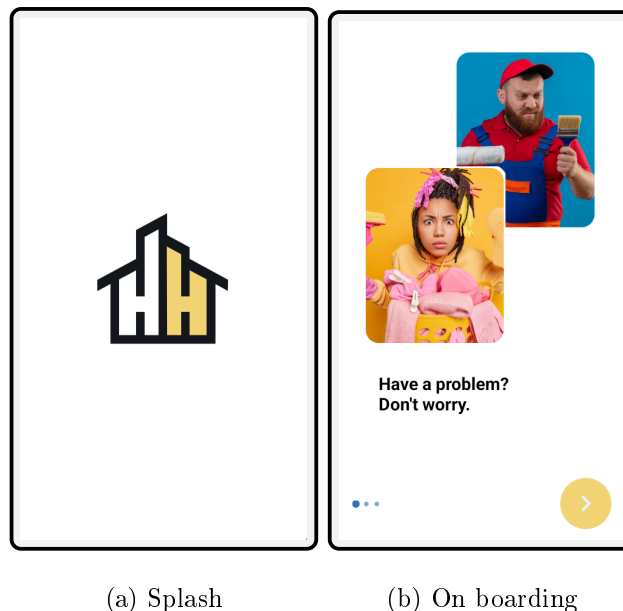
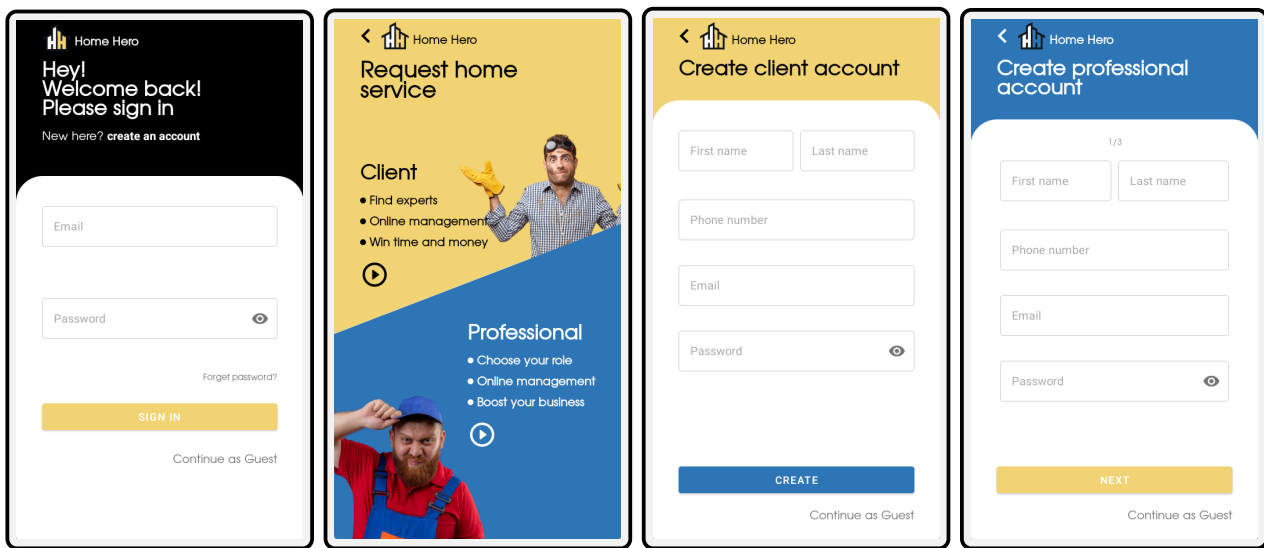


FIGURE 4.3 – Interfaces de lancement de l'application

4.4.2 Authentification

Après l'étape 4.3, si l'utilisateur n'est pas authentifié, l'interface de connexion (4.4a) sera affichée. Dans cette interface, l'utilisateur a la possibilité d'ouvrir sa session en utilisant son compte existant, de créer un nouveau compte ou de continuer en tant que visiteur. Pour créer un compte, il doit sélectionner

son rôle dans l'interface (4.4b) et remplir le formulaire correspondant (4.4c) ou (4.4d) en fournissant les informations requises.



(a) Connexion

(b) Choisir rôle

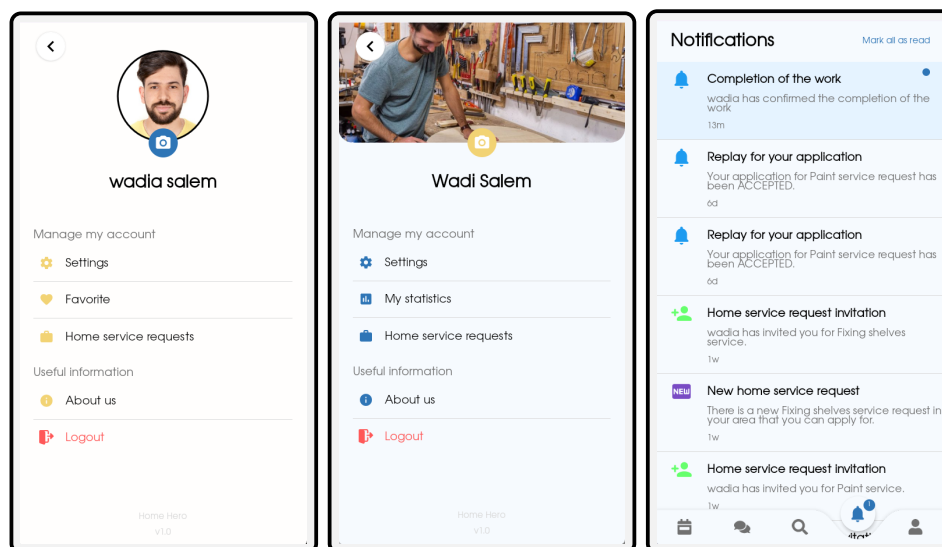
(c) Compte client

(d) Compte professionnel

FIGURE 4.4 – Interfaces d'authentification

4.4.3 Gestion profile

Pour la gestion du profile, le client (4.5a) ou le professionnel (4.5b) dispose d'une interface unique où il peut effectuer plusieurs actions. Il peut changer l'image de son profil, apporter des modifications à son compte, consulter ses demandes de services, accéder aux informations sur notre application et fermer son compte. Le client dispose également d'une liste de favoris, comme illustré dans la figure 4.6, tandis que le professionnel a accès à une section dédiée aux statistiques, détaillée dans la figure 4.7. Notre application garantit à l'utilisateur d'être tenu informé en recevant différentes notifications, comme indiqué dans l'interface (4.5c). Lorsque l'utilisateur clique sur une notification, il sera dirigé vers l'interface associée à cette notification.



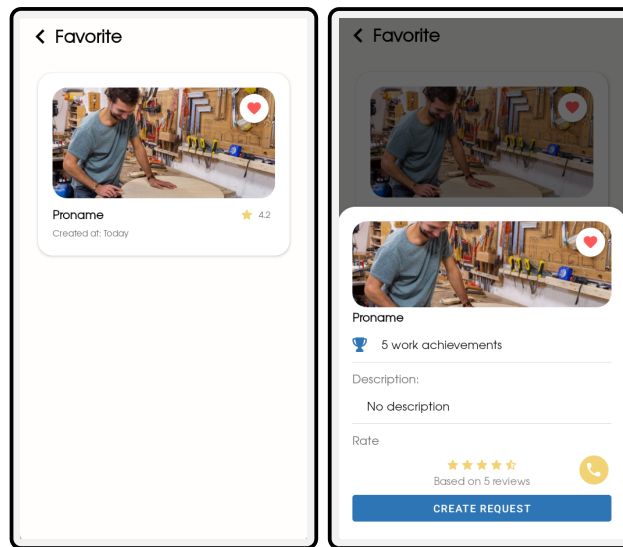
(a) Profile client

(b) Profile professionnel

(c) Notifications

FIGURE 4.5 – Interfaces de gestion profile

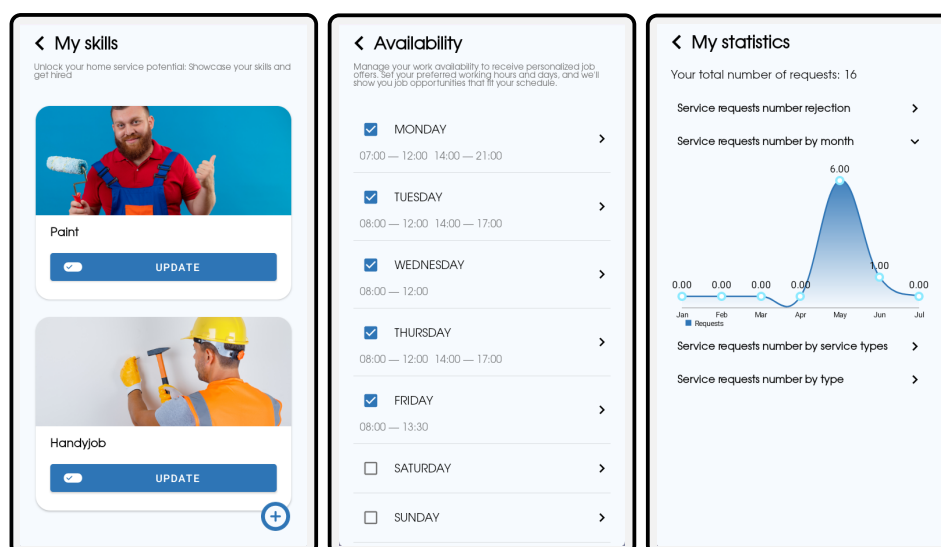
Il arrive parfois que le client souhaite enregistrer des professionnels afin de pouvoir y accéder facilement ultérieurement. L'interface (4.6a) représente les professionnels ajoutés à sa liste des favoris. Après avoir sélectionné un professionnel, le client peut créer une nouvelle demande privée et consulter davantage des informations telles que sa description, le nombre de services accomplis, les avis des clients et son numéro de contact pour l'appeler.



(a) Favoris (b) Information

FIGURE 4.6 – Interfaces des favoris

Le professionnel dispose d'une interface (4.7a) dédiée à la gestion de ses compétences, où il peut ajouter, modifier ou supprimer celles-ci. Le professionnel ne reçoit que les demandes de service qu'il peut réaliser en fonction de ses compétences ajoutées. De même, pour ses disponibilités, le professionnel gère son agenda de disponibilité (4.7b) afin de fixer ses horaires de travail. Enfin, il peut accéder à la partie statistiques concernant ses services pour visualiser des graphiques et des données pertinentes, tels que le nombre total des services, le ratio d'acceptation par rapport aux refus, le nombre de services pour chaque mois, le nombre de services par type de service et le nombre de services par visibilité (publique/privée).



(a) Compétences (b) Disponibilité (c) Statistiques

FIGURE 4.7 – Interfaces de gestion profile professionnel

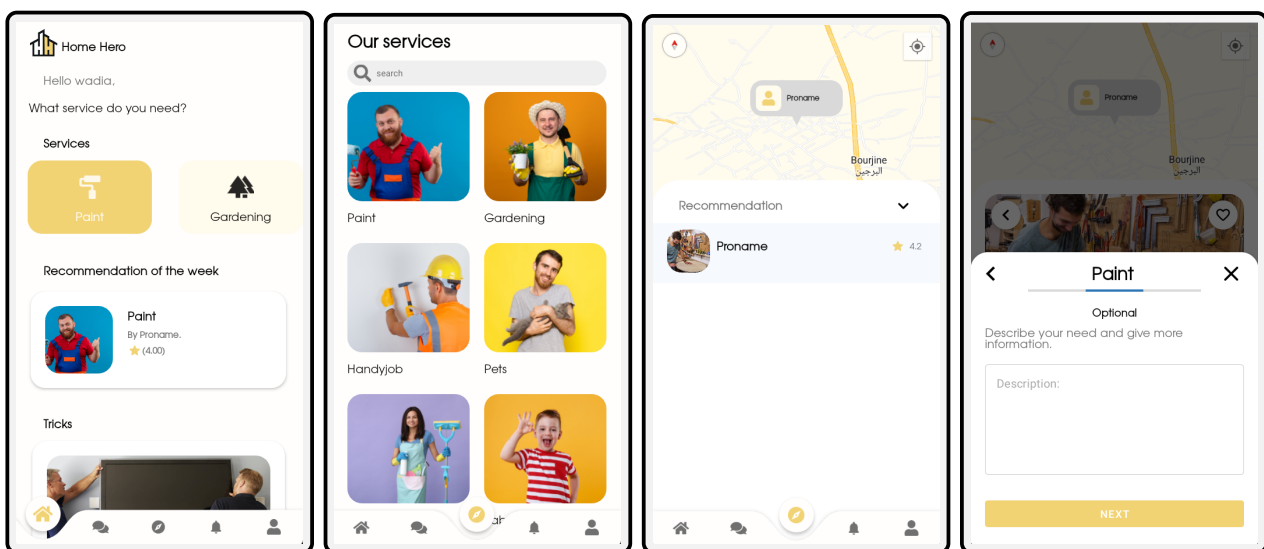
4.4.4 Demander un service

Nous avons développé plusieurs options pour que le client puisse demander un service. Tout d'abord, à partir de l'interface d'accueil (4.8a), le client dispose de plusieurs possibilités :

- Un accès rapide aux types de services pour choisir entre publier une demande ou consulter la liste des professionnels ayant les compétences requises pour le type de service sélectionné.
- Des recommandations de professionnels basées sur les sept derniers jours pour chaque type de service. En cliquant sur une recommandation, le formulaire pour envoyer une nouvelle demande privée au professionnel recommandé avec le type de service s'ouvre.
- Les astuces de bricolage partagées sont en cours de développement.

Ensuite, l'interface des types de services (4.8b) permet au client de sélectionner le type de service et de choisir d'afficher la liste des professionnels correspondants ou de publier sa demande. L'interface (4.8c) représente une carte géographique qui affiche les professionnels triés selon notre système de recommandation, avec la possibilité d'interagir en zoomant, dézoomant et en sélectionnant des professionnels via la carte.

Enfin, le formulaire (4.8d) s'affiche après avoir choisi un professionnel pour une demande privée ou après avoir choisi de publier la demande lors de la sélection du type de service pour une demande publique. Dans ce formulaire, le client peut saisir une description, ajouter des images, indiquer la date, l'heure et l'adresse du travail.



(a) Accueil client

(b) Types des services

(c) Professionnels

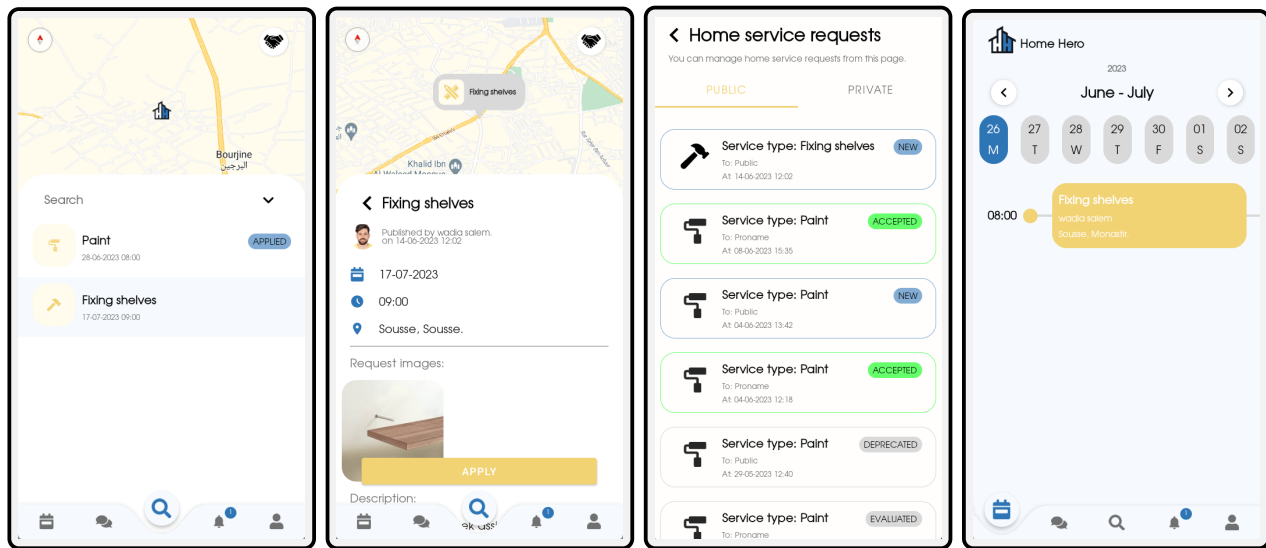
(d) Formulaire

FIGURE 4.8 – Interfaces de demander un service

4.4.5 Gestion des demandes des clients

- Le professionnel recherche les demandes publiques à partir de l'interface (4.9a) . Il peut sélectionner une demande soit sur la carte géographique, soit dans la liste fournie. Une fois qu'il a choisi une demande, les détails de la demande sélectionnée s'affichent. Ensuite, le professionnel a la possibilité de proposer ses services en cliquant sur le bouton "Apply" (4.9b), ce qui lui permettra d'envoyer directement sa proposition au client, qui est le demandeur de service. Il convient de noter que les demandes auxquelles le professionnel accède sont de nature publique, c'est-à-dire qu'elles sont visibles et ouvertes à tous les professionnels.
- Concernant les demandes privées, le professionnel peut les consulter de deux manières. Premièrement, en cliquant sur le bouton situé en haut à droite de l'interface (4.9a). Deuxièmement, il peut accéder aux demandes privées via son profil en choisissant l'option de visualisation des demandes privées, comme indiqué dans l'interface (4.9c).

- L'interface (4.9d) représente le calendrier du professionnel pour la gestion des demandes qu'il a acceptées.



(a) Demandes publiques

(b) Proposer service

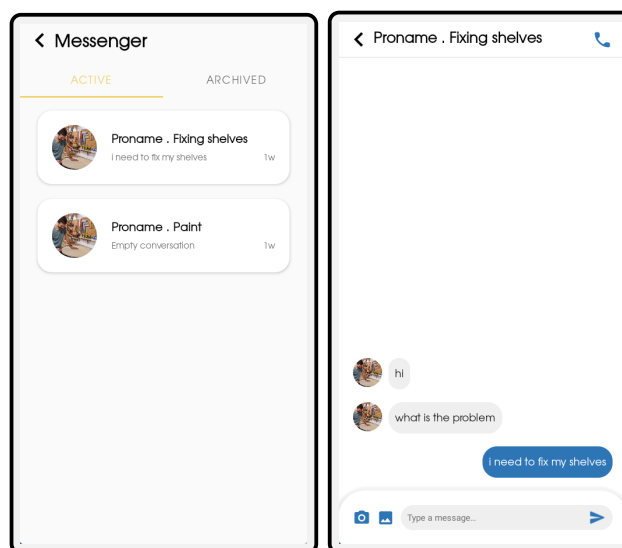
(c) Demandes

(d) Calendrier

FIGURE 4.9 – Interfaces de gestion demandes des clients

4.4.6 Messagerie

Nous avons développé un module de messagerie utilisant Socket.IO qui permet aux utilisateurs d'avoir des conversations en temps réel. L'interface (4.10a) représente la liste paginée des conversations de l'utilisateur. Chaque conversation est dédiée à une demande de service et sera archivée une fois le service terminé. En cliquant sur une conversation, la liste des messages s'affiche, offrant la possibilité d'envoyer des messages sous forme de texte ou d'images. De plus, l'interface permet également aux utilisateurs d'effectuer des appels téléphoniques en cliquant sur l'option d'appel pour communiquer directement avec le destinataire de la conversation.



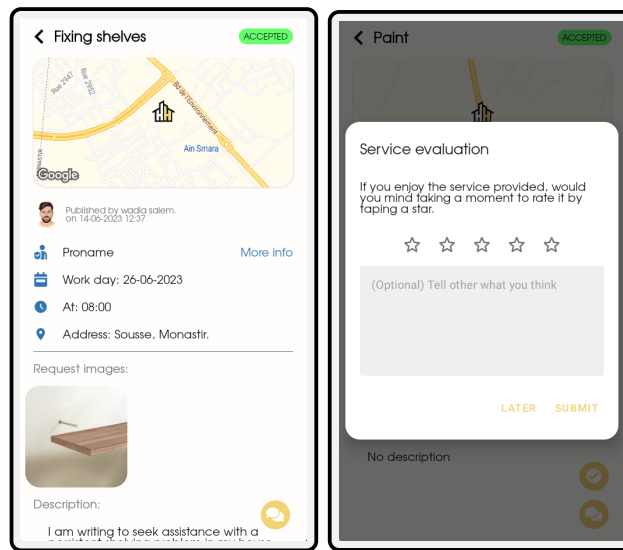
(a) Messages

(b) Conversation

FIGURE 4.10 – Interfaces de messagerie

4.4.7 Évaluation

Après la fin du service, le client accède à l'interface de la demande (4.11a) via son profil et confirme la fin du travail. Ensuite, une interface de dialogue (4.11) s'affiche, permettant au client de donner son avis et de fournir une évaluation du service reçu.



(a) Demande

(b) Évaluation

FIGURE 4.11 – Interfaces d'évaluation

4.4.8 Administration

En ce qui concerne l'aspect administratif, nous avons mis en place un tableau de bord web pour le contrôle de l'application. la figure 4.12 représente l'interface d'authentification d'administrateur.

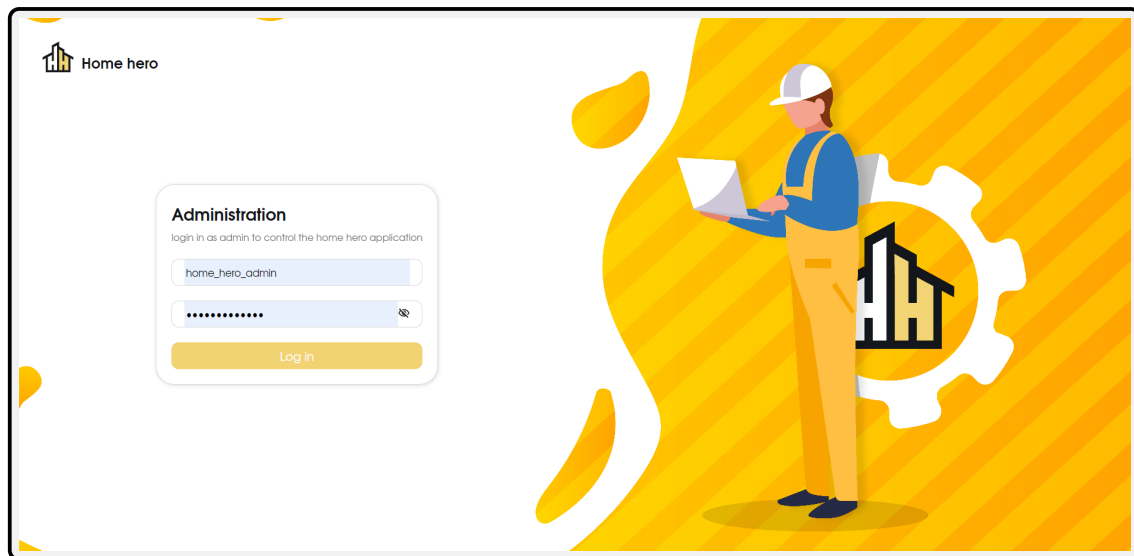


FIGURE 4.12 – Interface d'authentification d'administration

L'administrateur dispose d'un accès privilégié lui permettant de consulter les graphiques statistiques, la liste des utilisateurs, les différents types de services proposés, ainsi que les réclamations reçues. Il est important de noter que la fonctionnalité de gestion des réclamations est en cours de développement et sera prochainement disponible. Cette fonctionnalité permettra à l'administrateur de gérer et de traiter les réclamations des utilisateurs de manière efficace.

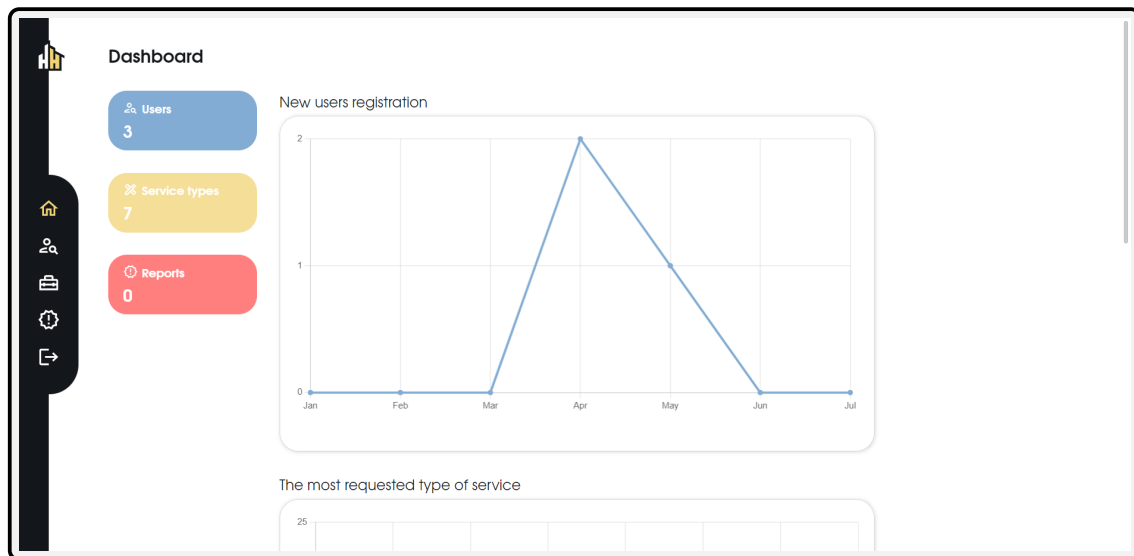


FIGURE 4.13 – Interface d'accueil d'administration

L'administrateur a accès à la liste des utilisateurs de l'application sous la forme d'un tableau, ce qui lui permet de gérer ces utilisateurs. Il peut également consulter plus d'informations concernant un utilisateur spécifique et choisir de le bloquer ou de le débloquer selon les besoins.

Users
You can manage home hero users from this interface.

ID	First name	Last name	Email	Role	Is blocked	Action
1	Wadia	SALEM	wadi.selem1045@gmail.com	CLIENT	No	Details
2	Wadi	SALEM	wadi.selem104@gmail.com	PRO	No	Details
6	Wadia	SALEM	wadi.selem1@gmail.com	PRO	No	Details

1 row selected

Rows per page: 5 1-3 of 3

FIGURE 4.14 – Interface des utilisateurs d'administration

Pour la gestion des types de service, l'interface représentée dans la Figure 4.15 illustre les fonctionnalités d'ajout, de modification et de suppression.

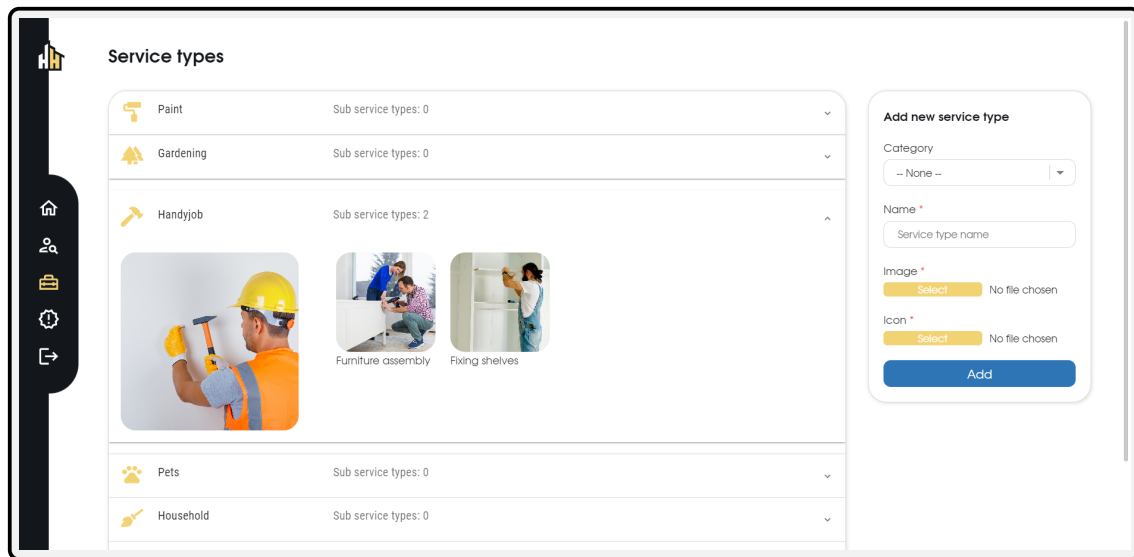


FIGURE 4.15 – Interface des types de service d'administration

Conclusion

Tout au long de ce dernier chapitre, nous avons présenté l'environnement matériel et logiciel nécessaire pour l'implémentation du logiciel. Nous avons également décrit l'architecture physique à l'aide d'un diagramme de déploiement, mettant en évidence les différents composants matériels et logiciels utilisés dans notre plateforme. De plus, nous avons partagé les résultats de notre développement en fournissant des captures d'écran de quelques interfaces clés de notre plateforme. Ces captures d'écran permettent de visualiser et d'illustrer les différentes fonctionnalités et interfaces utilisateur que nous avons développées.

Conclusion générale

Le présent rapport décrit en détail les différentes étapes suivies pour la mise en place d'une plateforme de réseau professionnel. Notre logiciel a été conçu pour être utilisé par trois types d'acteurs : les visiteurs qui consultent nos services et nos prestataires, les clients qui partagent leurs demandes de services de dépannage à domicile, et les professionnels qui fournissent ces services. Notre application offre un confort accru, des gains de temps et d'argent appréciables pour tous les utilisateurs.

Le logiciel développé répond efficacement aux besoins du client. Les fonctionnalités actuellement mises en œuvre fonctionnent parfaitement et satisfont pleinement les attentes. Tout d'abord, le logiciel offre une gestion du profil robuste, permettant aux utilisateurs de créer et de gérer facilement leur compte. Ils peuvent ajouter des informations personnelles et personnaliser leur profil selon leurs préférences. De plus, l'application offre un contrôle complet aux utilisateurs. Ils peuvent facilement naviguer à travers les différentes sections de l'application, accéder aux fonctionnalités disponibles et les utiliser de manière intuitive. En outre, le logiciel offre également une gestion efficace des services demandés par les clients. Ils peuvent spécifier leurs besoins, effectuer des recherches pour trouver des services correspondants, et bénéficier d'un processus de demande fluide.

Ce stage a été une expérience enrichissante tant sur le plan technique que professionnel. Sur le plan technique, ce projet m'a permis d'explorer de nouveaux domaines informatiques, tels que le développement d'applications Android natives. Sur le plan professionnel, ce projet m'a offert l'opportunité de vivre une expérience de travail au sein d'une entreprise. Cela a été une véritable occasion de mettre en pratique mes compétences d'ingénieur de modélisation et de développement.

En conclusion, les fonctionnalités de notre logiciel répondent aux besoins fonctionnels et non fonctionnels de nos clients, bien que certaines parties soient encore en cours de développement, telles que le module intelligent de vérification des données, le système de réclamation, les badges d'excellence et la fonctionnalité de partage d'astuces de bricolage. Cependant, nous envisageons d'ajouter prochainement de nouvelles fonctionnalités, telles que la généralisation de notre système de recommandation pour couvrir les professionnels et leur fournir des recommandations d'amélioration de leurs services basées sur les évaluations des clients, un service de chat-bot pour aider les professionnels à répondre à leurs messages, un autre rôle pour les entreprises qui gèrent plusieurs professionnels, un plan premium visant à garantir une source de revenus pour notre application, ainsi qu'un système de réalité augmentée pour les interventions à distance.

Références

- [1] Android runtime (art). <https://source.android.com/docs/core/runtime>. [Accès le 21 Juin 2023].
- [2] Android studio. <https://developer.android.com/studio>. [Accès le 20 Juin 2023].
- [3] Coil. <https://github.com/coil-kt/coil>. [Accès le 20 Juin 2023].
- [4] Coroutines. <https://kotlinlang.org/docs/coroutines-overview.html>. [Accès le 20 Juin 2023].
- [5] Dependency injection with hilt. <https://developer.android.com/training/dependency-injection/hilt-android>. [Accès le 20 Juin 2023].
- [6] Figma. <https://www.figma.com>. [Accès le 20 Juin 2023].
- [7] Firebase cloud messaging. <https://firebase.google.com/docs/cloud-messaging>. [Accès le 20 Juin 2023].
- [8] Flow. <https://kotlinlang.org/docs/flow.html>. [Accès le 20 Juin 2023].
- [9] Gitlab. <https://about.gitlab.com>. [Accès le 20 Juin 2023].
- [10] Google maps platform. <https://developers.google.com/maps/documentation>. [Accès le 20 Juin 2023].
- [11] Inversify. <https://www.inversify.io>. [Accès le 20 Juin 2023].
- [12] Kotlin. <https://www.kotlinlang.org>. [Accès le 20 Juin 2023].
- [13] Laragon. <https://laragon.org/docs/>. [Accès le 22 Juin 2023].
- [14] Pytesseract. <https://pypi.org/project/pytesseract/>. [Accès le 26 Juin 2023].
- [15] React. <https://react.dev/learn>. [Accès le 26 Juin 2023].
- [16] Retrofit. <https://square.github.io/retrofit/>. [Accès le 20 Juin 2023].
- [17] Save data in a local database using room. <https://developer.android.com/training/data-storage/room>. [Accès le 20 Juin 2023].
- [18] Soketio. <https://www.socket.io>. [Accès le 20 Juin 2023].
- [19] Swagger. <https://www.swagger.io>. [Accès le 20 Juin 2023].
- [20] Typescript. <https://www.typescriptlang.org>. [Accès le 20 Juin 2023].
- [21] Visual studio code. <https://code.visualstudio.com>. [Accès le 20 Juin 2023].
- [22] Dr. Selma Belgacem. Les patrons d'architecture (2) clean architecture. 2021-2022.
- [23] Dr. Selma Belgacem. Processus de développement : Méthodologies non agiles. 2021-2022.
- [24] Régis Clouard. *Chapitre 3 : Catalogue de patrons de conception*.
- [25] Robert C. Martin. *Agile Software Development : Principles, Patterns, and Practices*. Prentice Hall, Upper Saddle River, NJ, 2003.
- [26] Robert C. Martin. The clean architecture. <http://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>, 2012. [Accès le 16 Juin 2023].
- [27] Tom Mitchell. Machine learning. <http://www.cs.cmu.edu/~tom/mlbook.html>, 1997. [Accès le 26 Juin 2023].